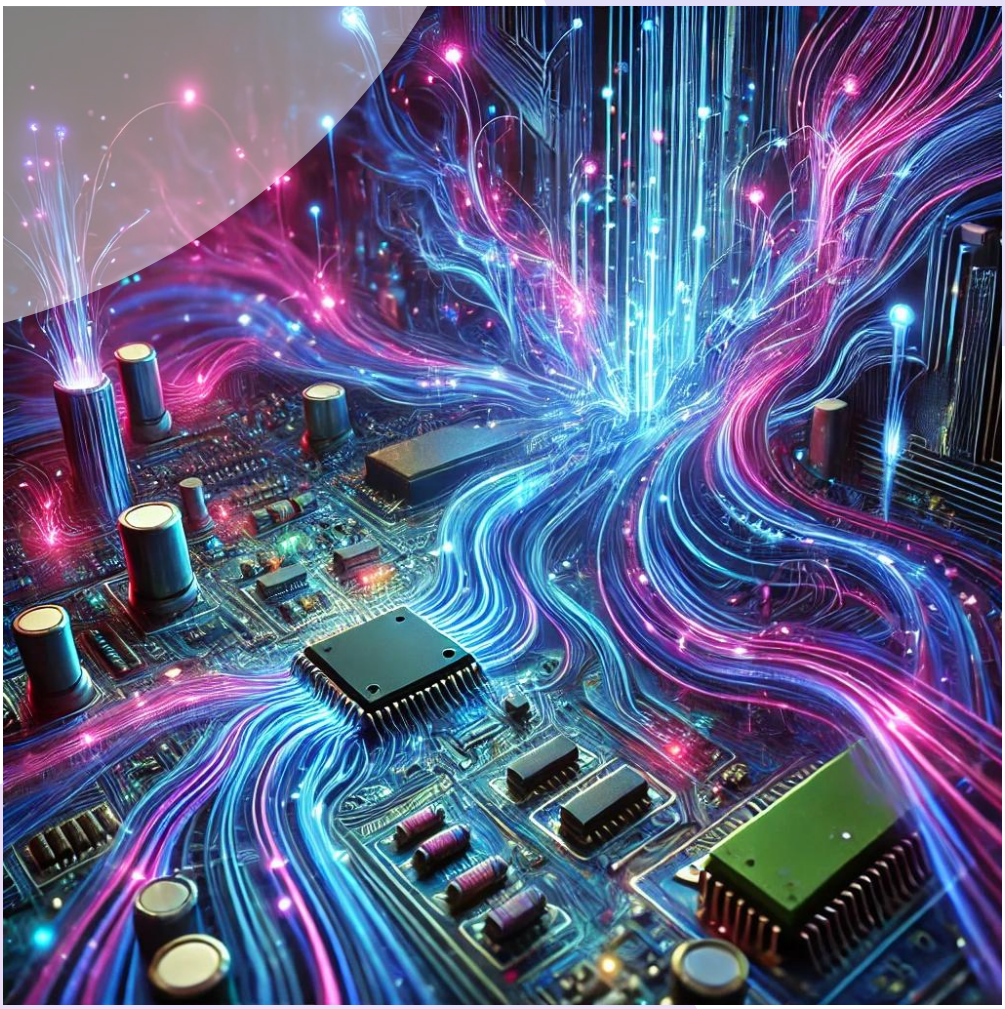


MINI PROJECT REPORT

Artificial Intelligence Course

TITLE:
Regression

Aylar Rahimzadeh
Fatemeh Sajed
آزمایشگاه الکت رونیک ۱ و۲

فهرست

Titanic dataset

۱. آشنایی مجموعه داده ها
۲. پیش پردازش داده ها
۳. انتخاب ویژگی ، آموزش و ارزیابی

New DataSet

۱. دریافت دادگان
۲. پاکسازی و پیش پردازش داده ها برای سطر اول
۳. آموزش های مدل رگرسیون برای سطر اول
۴. پاکسازی و پیش پردازش داده ها برای سطر دوم
۵. آموزش های مدل رگرسیون برای سطر دوم
۶. پاکسازی و پیش پردازش داده ها برای سطر سوم
۷. آموزش های مدل رگرسیون برای سطر سوم
۸. پاکسازی و پیش پردازش داده ها برای قاب داده شامل سطر اول و دوم
۹. آموزش های مدل رگرسیون برای قاب داده شامل سطر اول و دوم

لینک های مربوطه

تایتانیک

https://colab.research.google.com/drive/1WYfflUgf_mxfSCcqSdI-UksbTi6pQKOI?usp=sharing

سطر اول داده جدید:

https://colab.research.google.com/drive/1aQNIyhulOJQmIeu3_wrmGpiHT-2Rj061?usp=sharing

سطر دوم:

https://colab.research.google.com/drive/15oaiMjVNF2eWQEInn0qSwrxYu0q5dJRh?usp=sharing

سطر سوم:

https://colab.research.google.com/drive/1nI01Wc9nwLbBfG9bJN9cbLpcAdvE0E_o?usp=sharing

دیتافریم سطر۱و۲:

https://colab.research.google.com/drive/1LZd2VsUWt_Mdbajtsj4zHhVtOCcOjFvq?usp=sharing

۱. پیش پردازش داده ها

۱/۱. بررسی مجموعه داده ها

در این بخش به موارد زیر می پردازیم:

- **بدست اوردن ساختار کلی داده ها**
- **رسم نمودار correlation و بدست آوردن همبستگی ها**
- **رسم نمودار های پراکندگی و هگزبین**
- **رسم نمودار پراکندگی**

بدست آوردن ساختار کلی داده ها:

سطر اول کد این فایل مربوط به تمامی کتابخانه هایی است که در این تحلیل دیتاست مورد نیاز بوده است.

سطر بعدی در این فایل مربوط به خواندن دیتا می باشد که در متغیری ذخیره شده است تا بعدا بتوان از آن استفاده کرد و سپس برای نمایش آن با استفاده از تابع head() چند سطر ابتدایی را چاپ می کنیم.

سپس همان طور که در صورت سوال خواسته شده و مرسوم است باید اطلاعاتی کلی راجع به دیتایی که در حال تحلیل آن هستیم بدست بیاوریم بنابراین با استفاده از تابع describe() اطلاعاتی آماری از آن دریافت می کنیم. از مینیمم سطر ها و ستون ها بعدا در ادامه استفاده خواهد شد.

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

قدم بعدی آن است که درمورد نوع دیتای هر ستون تحقیق کنیم تا اگر دیتایی object است و قابلیت آن را دارد تا به یک اطلاعات آماری تبدیل شود را شناسایی کنیم بنابراین با استفاده از تابع detypes، آبجکت ها را شناسایی می کنیم.

```
df.dtypes
```

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

نکته مهم!!!!: در روند صورت سوال این گونه آمده است که ابتدا نمودار هایی رسم شود و سپس داده ها را پاک سازی کنیم اما از انجایی که گروه ها همفکری کرد که بهتر است داده های NAN ابتدا شناسایی شوند و داده کلیر شود و سپس به ادامه تحلیل بپردازیم همینجا داده های NAN را شناسایی کرده و درموردشان تصمیم میگیریم و سپس به رسم نمودار ها خواهیم پرداخت.

بنابراین باید داده های NAN را شناسایی کنیم برای این منظور از تابع isnull().sum() استفاده می کنیم که مجموع دیتا های not a number در هر ستون (ویژگی) به ما نشان می دهد.

```
: df.isnull().sum()
```

```
: PassengerId      0
  Survived         0
  Pclass           0
  Name             0
  Sex              0
  Age            177
  SibSp            0
  Parch            0
  Ticket           0
  Fare             0
  Cabin          687
  Embarked         2
  dtype: int64
```

خروجی نشان میدهد که ستون age , cabin دارای دیتاهای نامشخص هستند بنابراین برای آن ها باید چاره ای اندیشید تا تحلیل های آماری را خراب نکنند.؛ البته قبل از این index ها را تغییر می دهیم تا تعداد ستون های اضافه کمتر شوند.

```
df.rename(index=df['PassengerId'], inplace=True)
df.drop(['PassengerId'],axis=1,inplace=True)
df
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

پس از این کار داده های نامشخص را به سه روش جایگزین می کنیم این روش کاملا دلخواه است و می تواند جور دیگری هم درنظر گرفته شود.

ستون سن: مقدارهای گمشده رو با میانگین سن‌ها پر می شوند.

ستون قیمت بلیط ها: با **میانه‌ی قیمت‌ها** پر می شوند (دلیل انتخاب: میانه در مقابل داده‌های پرت مقاومت است).

ستون بندرگاه: با **مد (mode)** یعنی پرتکرارترین مقدار پر می شوند.

- ستون شماره کابین: بیشتر مقادیر Cabin به شکل حروف + عدد هستند)
- این خط فقط **حرف اول** یعنی deck مربوطه رو نگه می‌داره. مثلاً C23 → C
- ولی اگذ مقدار "Unknown" باشه، همونطور باقی می مانند.

قدم بعدی آن است که ستون های متنی را به روشی تبدیل کنیم به داده هایی که از لحاظ آماری بتوان مورد بررسی قرار داد پس با استفاده از one-hot encoding آن ها را به ستون های عددی تبدیل می کنیم.

قدم بعدی نرمال سازی ویژگی های عددی است.

چرا این کار را می کنیم؟

چون زمانیکه دامنه داده های عددی مورد بررسی بسیار متفاوت باشد مقایسه نمودار ها و ویژگی های آنها دشوار و با خطا خواهد بود. بنابراین با استفاده از تابع get dummies() می توانیم گروهبندی ها را مشخص کنیم.

اینجا با استفاده از StandardScaler استاندارد سازی می کنیم و انحراف معیار ۱ است تا در مدل ها تاثیر برابر باشد.

	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S	Cabin_B	Cabin_C	Cabin_D	Cabin_E	Cabin_F	Ca
1	0	3	Braund, Mr. Owen Harris	-0.592481	1	0	A/5 21171	-0.502445	True	False	True	False	False	False	False	False	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0.638789	1	0	PC 17599	0.786845	False	False	False	False	True	False	False	False	
3	1	3	Heikkinen, Miss. Laina	-0.284663	0	0	STON/O2. 3101282	-0.488854	False	False	True	False	False	False	False	False	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.407926	1	0	113803	0.420730	False	False	True	False	True	False	False	False	
5	0	3	Allen, Mr. William Henry	0.407926	0	0	373450	-0.486337	True	False	True	False	False	False	False	False	

	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S	Cabin_B	Cabin_C	Cabin_D	Cabin_E	Cabin_F	Cabin_G	Cabin_T	Cabin_Unknown
	-0.592481	1	0	A/5 21171	-0.502445	True	False	True	False	False	False	False	False	False	False	True
	0.638789	1	0	PC 17599	0.786845	False	False	False	False	True	False	False	False	False	False	False
	-0.284663	0	0	STON/O2. 3101282	-0.488854	False	False	True	False	False	False	False	False	False	False	True
	0.407926	1	0	113803	0.420730	False	False	True	False	True	False	False	False	False	False	False
	0.407926	0	0	373450	-0.486337	True	False	True	False	False	False	False	False	False	False	True

در اینجا می توانیم ستون اسم ها را حذف کنیم چون اطلاعات خاصی از آن استخراج نمی شود و هیچ تاثیر آماری نه در آزمون و ارزیابی دارد و نه در بررسی ها تاثیری دارد.

```
df=df.drop(['Name'],axis=1,inplace=True)
df
```

رسم نمودار correlation و بدست آوردن همبستگی ها

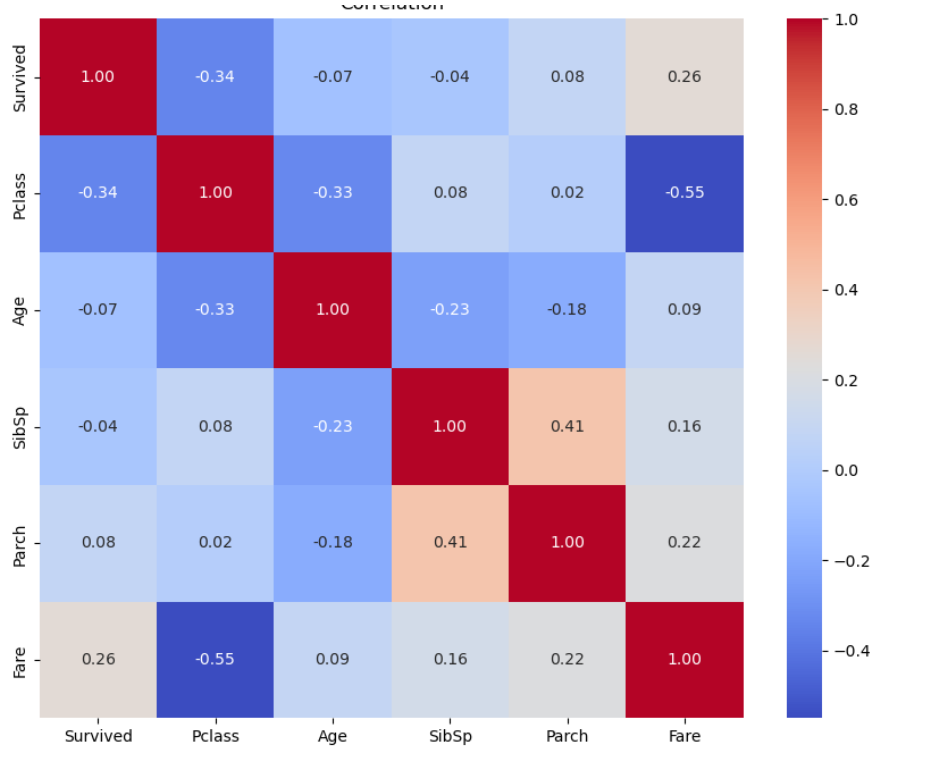
برای رسم نمودار همبستگی باید ابتدا داده های عددی را بدس بیاوریم برای اینکار از تابع select datatypes استفاده می کنیم درنتیجه یک دیتافریم داریم که فقط شامل ویژگی های عددی دیتای تایتانیک است. پس از آن با استفاده از تابع corr() ماتریس همبستگی را خساب می کنیم درواقع بین همه ستون ها ارتباط را نشان می دهد.

توضیحات:

مقدار هر سلول نشان می‌دهد که دو ویژگی عددی چقدر به هم وابسته هستند:

- **مقدار نزدیک به ۱: همبستگی مثبت قوی**
- **مقدار نزدیک به -۱: همبستگی منفی قوی**
- **مقدار نزدیک به ۰: بدون همبستگی**

سپس به سراغ رسم نمودار می رویم برای این کار از نمودار heatmap استفاده می کنیم و دلیل آن این ایست که می توان طیف ها را به خوبی با استفاده از رنگ ها نشان داد برای اینکار تنظیمات مربوطه مانند نمایش مقادیر داخل خانه های جدول و انتخاب پالت (تنالیته موردنظر برای نشان دادن رنگ ها) مشخص میکنیم.



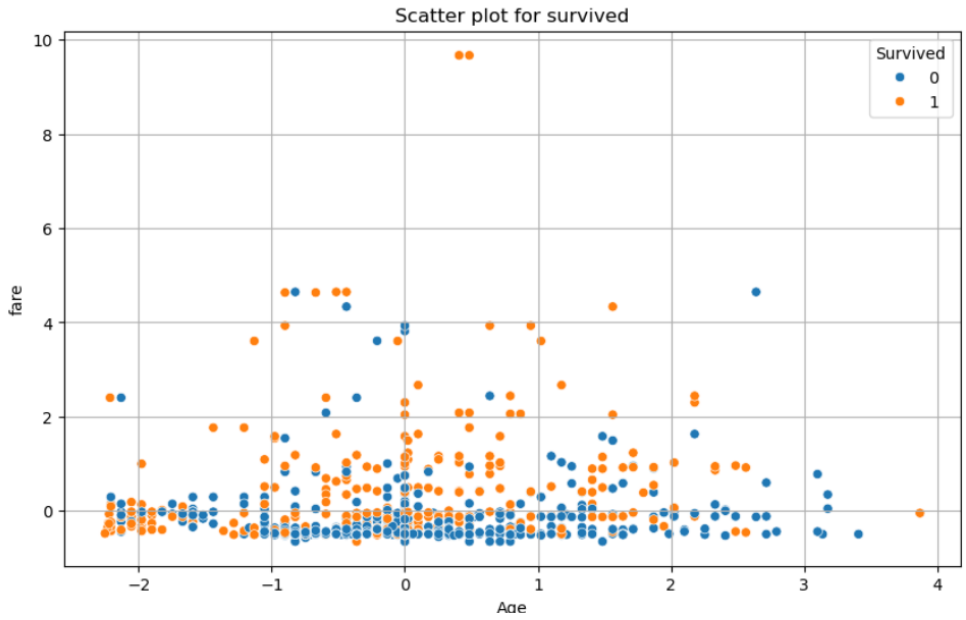
تحلیل خروجی:

ویژگی ۱	ویژگی ۲	ضریب همبستگی	نوع همبستگی	تفسیر
Survived	Pclass	-0.34	منفی متوسط	احتمال نجات در کلاس‌های بالاتر بیشتر بوده است.
Survived	Age	-0.07	تقریباً صفر	سن تأثیر قابل توجهی بر نجات نداشته است.
Survived	SibSp	-0.04	تقریباً صفر	تعداد خواهر/برادر همراه تأثیر کمی بر نجات داشته است.
Survived	Parch	0.08	تقریباً صفر	تعداد والدین/فرزندان همراه تأثیر کمی بر نجات داشته است.
Survived	Fare	0.26	مثبت ضعیف	پرداخت کرایه بیشتر با احتمال بیشتر نجات همراه بوده است.
Pclass	Age	-0.33	منفی متوسط	مسافران کلاس پایین‌تر معمولاً جوان‌تر بوده‌اند.
Pclass	SibSp	0.08	تقریباً صفر	رابطه خاصی بین کلاس سفر و خواهر/برادر دیده نمی‌شود.
Pclass	Parch	0.02	تقریباً صفر	رابطه خاصی بین کلاس سفر و والدین/فرزندان دیده نمی‌شود.
Pclass	Fare	-0.55	منفی نسبتاً قوی	کلاس پایین‌تر معمولاً با کرایه کمتر همراه بوده است.
Age	SibSp	-0.23	منفی ضعیف	افراد مسن‌تر معمولاً تنها سفر کرده‌اند.
Age	Parch	-0.18	منفی ضعیف	افراد مسن‌تر کمتر با والدین/فرزندان همراه بوده‌اند.
Age	Fare	0.09	تقریباً صفر	سن و کرایه پرداختی تقریباً ارتباطی ندارند.
SibSp	Parch	0.41	مثبت نسبتاً قوی	کسانی که خواهر/برادر داشته‌اند، احتمال همراهی با والدین/فرزندان بیشتر بوده است.

تعداد خواهر/برادر همراه، تا حد کمی با کرایه بیشتر همراه بوده است.	مثبت ضعیف	0.16	Fare	*SibSp*
تعداد والدین/فرزندان همراه، کمی با کرایه بیشتر همراه بوده است.	مثبت ضعیف	0.22	Fare	*Parch*

نمودار پراکندگی

```
[84]: # نمودار پراکندگی (scatter)
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="Age", y="Fare", hue="Survived")
plt.title("Scatter plot for survived")
plt.xlabel("Age")
plt.ylabel("fare")
plt.legend(title="Survived")
plt.grid(True)
plt.show()
```



تحلیل خروجی :

فراد نجات‌یافته (نقاط نارنجی) بیشتر در نواحی با کرایه‌های بالاتر قرار دارند، مخصوصاً در بازه سنی میانه.

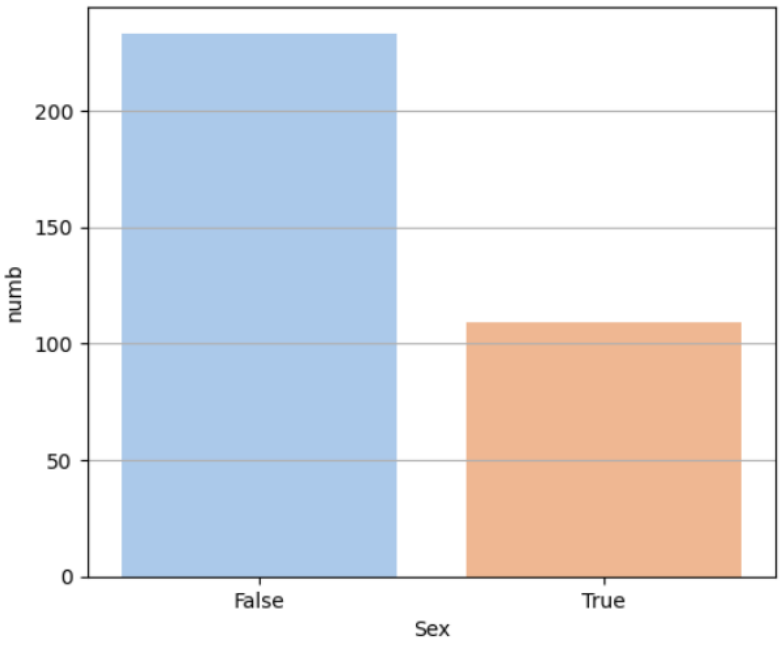
افراد نجات‌نیافته (نقاط آبی) بیشتر در نواحی با کرایه پایین متمرکز هستند.

سن (Age) تأثیر واضحی در نجات ندارد، چون در هر رنج سنی هم زنده‌مانده و هم فوت‌شده وجود دارد.

تراکم داده‌ها در سن نزدیک به میانگین (۰ پس از استانداردسازی) و کرایه نزدیک به ۰ بالاست (بیشتر افراد در طبقه متوسط یا پایین بوده‌اند)

رسم نمودار پراکندگی بر اساس جنسیت

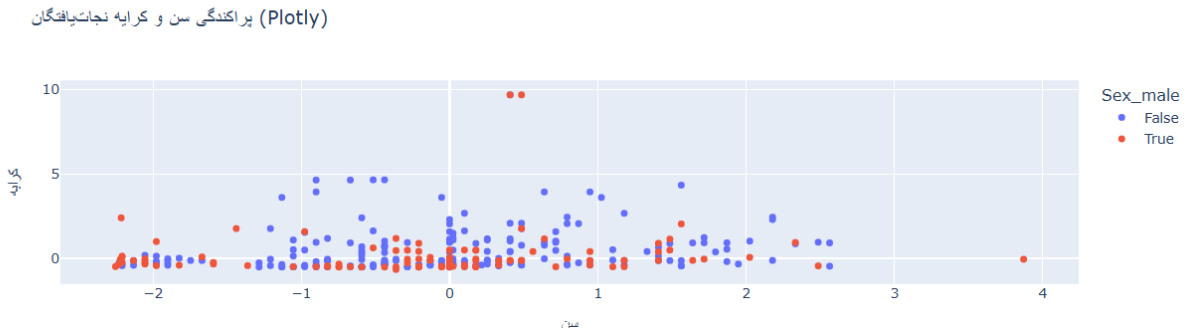
```
[15]: # 6. نمودار countplot برای جنسیت نجات‌یافتگان
plt.figure(figsize=(6, 5))
sns.countplot(data=survived_df, x="Sex_male", palette="pastel")
plt.xlabel("Sex")
plt.ylabel("numb")
plt.grid(True, axis='y')
plt.show()
```



رسم نمودار پراکندگی هزینه و سن

```
# Plotly نجات‌یافتگان
survived_df = df[df["Survived"] == 1]

fig = px.scatter(
    survived_df,
    x="Age",
    y="Fare",
    title="پراکندگی سن و کرایه نجات‌یافتگان (Plotly)",
    labels={"Age": "سن", "Fare": "کرایه"},
    hover_data=["Sex_male", "Pclass"],
    color="Sex_male"
)
fig.show()
```



تمرکز نقاط در کرایه پایین:

- بیشتر نجات‌یافتگان در بازه‌ی کرایه نزدیک به ۰ تا ۲ (بعد از استانداردسازی) قرار دارند.
- این نشان می‌دهد نجات صرفاً مربوط به افراد ثروتمند نبوده، هرچند در بخش کرایه‌های بالا نیز نجات‌یافته‌هایی دیده می‌شوند.

توزیع سنی متنوع:

- نجات‌یافتگان در تمام رده‌های سنی حضور دارند، اما تراکم بیشتری در اطراف میانگین سن (حدود ۰) وجود دارد.

مقایسه بر اساس جنسیت:

- تعداد بیشتری از نجات‌یافتگان را **زنان (نقاط آبی)** تشکیل می‌دهند.
- این موضوع با واقعیت تاریخی سیاست "اول زنان و کودکان" در تایتانیک تطابق دارد.

کرایه‌های بالا بیشتر توسط زنان پرداخت شده:

- در نواحی با کرایه بالا، اغلب نقاط به رنگ آبی هستند که نشان‌دهنده‌ی سهم بیشتر زنان در بین نجات‌یافتگان با بلیط‌های گران‌تر است.

۱/۲. تحلیل آماری مجموعه داده ها

تعداد اعضای خانواده

```
[ ]: #  محاسبه تعداد اعضای خانواده
df['FamilySize'] = df['SibSp'] + df['Parch']

# گروه‌بندی بر اساس اندازه خانواده و نرخ بقا
family_size_survival = df.groupby('FamilySize')['Survived'].mean()
#print("Impact of Family Size on Survival Rate:")
family_size_survival
```

```
[ ]: FamilySize
0     0.303538
1     0.552795
2     0.578431
3     0.724138
4     0.200000
5     0.136364
6     0.333333
7     0.000000
10    0.000000
Name: Survived, dtype: float64
```

توضیح:

SibSp: تعداد خواهر/برادر یا همسر همراه.

Parch: تعداد پدر/مادر یا فرزند همراه.

جمع این دو نشان‌دهنده تعداد اعضای خانواده همراه در سفر است.

گروه‌بندی داده‌ها بر اساس اندازه خانواده **(FamilySize)** و سپس محاسبه میانگین مقدار **Survived**برای هر گروه.

چون **Survived**فقط ۰ یا ۱ است، میانگین آن برابر با نرخ نجات آن گروه خواهد بود.

تحلیل خروجی:

خانواده‌های کوچک (۱ تا ۳ نفر) بیشترین شانس نجات را داشته‌اند.

افراد تنها یا خانواده‌های خیلی بزرگ نرخ نجات پایین‌تری داشته‌اند.

می‌توان نتیجه گرفت که در شرایط بحرانی (مثل تایتانیک)، وجود حداقل یک همراه مفید است، اما همراهی بیش از حد باعث کاهش شانس نجات می‌شود، احتمالاً به دلیل دشواری نجات گروه‌های بزرگ‌تر.

بررسی تاثیر سفر تنهایی:

```
# تأثیر سفر تنها یی
df['IsAlone'] = df['FamilySize'].apply(lambda x: 1 if x == 0 else 0)
alone_survival_rate = df.groupby('IsAlone')['Survived'].mean()
alone_survival_rate
```

```
IsAlone
0    0.505650
1    0.303538
Name: Survived, dtype: float64
```

توضیح:

برای این قسمت نیز زمانی که شماره خانواده ۱ باشد انرا تنها می نامیم پس باید میزان نجات یافتگانی که شماره خانواده شان ۱ است میانگین گیری شود.

```
#اضافه کردن شانس بقا  بر اساس اندازه خانواده
df['FamilySizeSurvivalRate'] = df['FamilySize'].map(family_size_survival)

#: اضافه کردن شانس بقا  بر اساس سفر تنها یی
df['AloneSurvivalRate'] = df['IsAlone'].map(alone_survival_rate)

# اضافه کردن شانس بقا  بر اساس گروه سنی
df['AgeGroupSurvivalRate'] = df['AgeGroup'].map(age_group_survival)
print(df[['FamilySize', 'FamilySizeSurvivalRate', 'IsAlone', 'AloneSurvivalRate', 'AgeGroup', 'AgeGroupSurvivalRate']].head())
```

```
   FamilySize  FamilySizeSurvivalRate  IsAlone  AloneSurvivalRate  \
1           1                0.552795        0           0.505650
2           1                0.552795        0           0.505650
3           0                0.303538        1           0.303538
4           1                0.552795        0           0.505650
5           0                0.303538        1           0.303538

      AgeGroup AgeGroupSurvivalRate
1         teen             0.344168
2  Yuong Adult             0.404255
3         teen             0.344168
4  Yuong Adult             0.404255
5  Yuong Adult             0.404255
```

نتیجه گیری: افراد تنها شانس بقای بالایی ندارند.

دسته young adult نرخ بالای بقای بیشتری نسبت به نوجوانان دارند. ولی به صورت کلی سنین مختلف نرخ بقای متفاوتی دارند.

۲. پیش پردازش داده ها

همانطور که قبل تر توضیح داده شد اسکیل کردن و نرمال سازی و جایگزینی داده های نامشخص در قسمت قبل و در ابتدا صورت گرفته است اینجا صرفا به سوالات توضیحی پاسخ داده خواهد شد.

تفاوت ویژگی‌های عددی و دسته‌ای

نوع ویژگی	تعریف	مثال‌ها	نوع داده در پایتون
عددی (Numerical)	ویژگی‌هایی با مقادیر عددی که امکان انجام عملیات ریاضی روی آن‌ها وجود دارد.	Fare ،Age	int, float
دسته‌ای (Categorical)	ویژگی‌هایی که نمایانگر طبقه‌ها یا گروه‌های غیرعددی هستند.	Embarked ،Sex Cabin	object، string

نوع ویژگی‌ها در داده‌های مجموعه تایتانیک

نام ویژگی	نوع ویژگی	نوع داده در پایتون	توضیحات
Age	عددی	float64	نیاز به پر کردن داده‌های گمشده و استانداردسازی دارد
Fare	عددی	float64	نیاز به پر کردن داده‌های گمشده و استانداردسازی دارد
Sex	دسته‌ای (متنی)	object	نیاز به رمزگذاری (One-Hot Encoding) دارد
Embarked	دسته‌ای (متنی)	object	پر کردن مقادیر گمشده با مد و سپس رمزگذاری
Cabin	دسته‌ای (متنی)	object	استخراج حرف اول کابین و رمزگذاری
Pclass	عددی با ماهیت طبقه‌ای	int64	بسته به تحلیل ممکن است به صورت دسته‌ای در نظر گرفته شود

پیش‌پردازش مناسب برای ویژگی‌های دسته‌ای

نوع پیش‌پردازش	توضیح
پر کردن مقادیر گمشده	معمولاً با مد (پرتکرارترین مقدار) انجام می‌شود.
رمزگذاری دسته‌ای (Encoding)	استفاده از One-Hot Encoding یا Label Encoding برای تبدیل مقادیر متنی به عددی.

مقایسه نرمال‌سازی و استانداردسازی

روش	تعریف	دامنه خروجی	کاربرد مناسب
نرمال‌سازی (Normalization)	تبدیل مقادیر به بازه[0, 1]	[0, 1]	برای داده‌هایی با مقیاس متفاوت یا توزیع غیرنرمال
استانداردسازی (Standardization)	تنظیم میانگین به صفر و انحراف معیار به یک	بدون دامنه مشخص	برای مدل‌هایی حساس به مقیاس مانندSVM ، KNN

مدل رگرسیون خطی به مقیاس ویژگی ها نیز حساس است بنابراین استانداردسازی نیاز است. کاری که قبلا انجام شده است.

۳. آزمون و ارزیابی و انتخاب ویژگی

ایجاد ویژگی جدید

در این تحلیل، ابتدا ویژگی‌های جدیدی به مجموعه داده افزوده شده‌اند. هدف از این مرحله، استخراج اطلاعات ترکیبی از ویژگی‌های موجود برای تقویت عملکرد مدل یادگیری ماشین است. ویژگی اول، نسبت سن به کرایه (Age_Fare_Ratio) است که ممکن است نشان‌دهنده نوع سفر یا سطح اقتصادی باشد. ویژگی دوم، اندازه خانواده (Family_Size) است که با جمع تعداد خواهر/برادر و والدین/فرزندان به‌دست می‌آید. ویژگی سوم، متغیر Is_Aloneاست که مشخص می‌کند آیا مسافر به‌تنهایی سفر کرده یا با همراه بوده است.

در مرحله بعد، الگوریتم رگرسیون لاسو (Lasso Regression) برای انتخاب ویژگی‌ها استفاده شده است. ابتدا داده‌ها آماده‌سازی شده‌اند و ستون‌های غیرضروری مانند متغیر هدف(Survived) ، شماره بلیت، و دسته‌بندی سنی حذف شده‌اند. پس از تقسیم داده‌ها به مجموعه‌های آموزش و آزمون، مدل لاسو با استفاده از اعتبارسنجی متقابل آموزش دیده است. لاسو با استفاده از جریمه L1 ، برخی ویژگی‌ها را حذف کرده و تنها ویژگی‌های مؤثر را حفظ می‌کند. ویژگی‌هایی که ضریب آن‌ها صفر نشده است، به‌عنوان ویژگی‌های مهم در نظر گرفته می‌شوند.

در مرحله سوم، روش RFE (حذف بازگشتی ویژگی‌ها) با استفاده از مدل پایه رگرسیون لجستیک اجرا شده است. این روش به‌صورت مرحله‌ای، ضعیف‌ترین ویژگی‌ها را حذف می‌کند تا تنها تعداد مشخصی از بهترین ویژگی‌ها باقی بمانند. در اینجا پنج ویژگی انتخاب شده‌اند که بیشترین تأثیر را بر پیش‌بینی متغیر هدف داشته‌اند.

در پایان، اشتراک بین ویژگی‌های انتخاب‌شده توسط دو روش لاسو و RFE گرفته شده و به‌عنوان ویژگی‌های نهایی مؤثر معرفی شده‌اند. این تقاطع کمک می‌کند تا تنها متغیرهایی که از نظر هر دو روش دارای اهمیت هستند، برای مدل نهایی مورد استفاده قرار گیرند. این روش ترکیبی به بهبود دقت مدل و کاهش پیچیدگی آن کمک می‌کند.

خروجی:

```
Features selected by Lasso: Index(['Pclass', 'Age', 'Parch', 'Fare', 'Sex_male', 'Embarked_S', 'Cabin_B',
       'Cabin_C', 'Cabin_D', 'Cabin_E', 'Cabin_F', 'Cabin_G', 'Cabin_Unknown',
       'FamilySize', 'IsAlone', 'FamilySizeSurvivalRate', 'Age_Fare_Ratio',
       'Family_Size', 'Is_Alone'],
      dtype='object')
Features selected by RFE: Index(['Pclass', 'Sex_male', 'Cabin_E', 'Cabin_F', 'FamilySizeSurvivalRate'], dtype='object')
Final selected features: ['Cabin_E', 'Pclass', 'Cabin_F', 'Sex_male', 'FamilySizeSurvivalRate']
```

توضیح:

۱ .ویژگی‌های انتخاب‌شده توسط لاسو(Lasso)

مدل لاسو ویژگی‌هایی را انتخاب کرده که ضرایب آن‌ها صفر نبوده‌اند. این ویژگی‌ها شامل موارد زیر هستند:

- ویژگی‌های عددی: Pclass، Age، Parch، Fare
- ویژگی‌های باینری یا رمزگذاری‌شده: Sex_male، Embarked_S
- ویژگی‌های مربوط به کابین: Cabin_B، Cabin_C، Cabin_D، Cabin_E، Cabin_F، Cabin_G، Cabin_Unknown
- ویژگی‌های مهندسی‌شده: FamilySize، IsAlone، FamilySizeSurvivalRate، Age_Fare_Ratio، Is_Alone، Family_Size

این انتخاب نشان می‌دهد که مدل لاسو بسیاری از متغیرهای خام و مهندسی‌شده را مؤثر در پیش‌بینی در نظر گرفته است.

۲. ویژگی‌های انتخاب‌شده توسط RFE

روش RFE تنها پنج ویژگی برتر را انتخاب کرده است:

- Pclass
- Sex_male
- Cabin_E
- Cabin_F
- FamilySizeSurvivalRate

این ویژگی‌ها با استفاده از مدل رگرسیون لجستیک و ارزیابی تکراری انتخاب شده‌اند. ویژگی‌های انتخاب‌شده شامل اطلاعات مکانی (کابین)، اجتماعی (جنسیت و طبقه سفر) و آماری (نرخ بقا براساس اندازه خانواده) هستند.

۳. ویژگی‌های نهایی (مشترک بین دو روش)

ویژگی‌هایی که هم توسط لاسو و هم توسط RFE انتخاب شده‌اند، به‌عنوان ویژگی‌های نهایی برای مدل‌سازی در نظر گرفته شده‌اند:

- Pclass
- Sex_male
- Cabin_E

- Cabin_F
- FamilySizeSurvivalRate

این ویژگی‌ها از نظر دو الگوریتم مستقل، بیشترین قدرت پیش‌بینی برای متغیر هدف (Survived) را دارند و می‌توانند به عنوان ورودی نهایی مدل انتخاب شوند.

رگرسیون باینری و تحلیل های آن

کد:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

# ایجاد مدل رگرسیون لجستیک
log_reg = LogisticRegression(max_iter=1000)

# ------------------- 1. مدل با ویژگی‌های انتخابی توسط لاسو -------------------
X_lasso = X[lasso_selected_features]  # ویژگی‌های انتخاب شده توسط لاسو
lasso_score = cross_val_score(log_reg, X_lasso, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with Lasso selected features: {lasso_score:.4f}")

# ------------------- 2. مدل با ویژگی‌های انتخابی توسط RFE -------------------
X_rfe = X[rfe_selected_features]  # ویژگی‌های انتخاب شده توسط RFE
rfe_score = cross_val_score(log_reg, X_rfe, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with RFE selected features: {rfe_score:.4f}")

# ------------------- 3. مدل با ویژگی‌های انتخابی توسط هر دو روش -------------------
X_final = X[final_selected_features]  # ویژگی‌های انتخاب شده توسط هر دو روش
final_score = cross_val_score(log_reg, X_final, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with final selected features (intersection of Lasso and RFE): {final_score:.4f}")
```

```
Mean accuracy with Lasso selected features: 0.8081
Mean accuracy with RFE selected features: 0.7935
Mean accuracy with final selected features (intersection of Lasso and RFE): 0.7935
```

توضیح:

در این بخش، سه مدل **رگرسیون لجستیک** با استفاده از سه مجموعه مختلف از ویژگی‌ها آموزش داده شده‌اند تا عملکرد آن‌ها از نظر دقت (accuracy) مقایسه شود. ارزیابی مدل‌ها با استفاده از **اعتبارسنجی متقابل پنج‌تایی** انجام شده است.

۱. **ساخت مدل پایه**

- یک نمونه از رگرسیون لجستیک ساخته شده با تعداد تکرارهای بیشتر از حالت پیش‌فرض (برای اطمینان از همگرایی مدل).

۲. **مدل‌سازی با ویژگی‌های انتخاب‌شده توسط لاسو**

- ابتدا زیرمجموعه‌ای از داده‌ها شامل فقط ویژگی‌هایی که توسط لاسو انتخاب شده‌اند، ایجاد شده است.
- سپس دقت مدل با استفاده از اعتبارسنجی متقابل محاسبه شده و میانگین آن ثبت شده است.

۳. **مدل‌سازی با ویژگی‌های انتخاب‌شده توسط RFE**

- مشابه مرحله قبل، اما با استفاده از ویژگی‌هایی که توسط روش حذف بازگشتی (RFE) انتخاب شده‌اند.
- دقت میانگین این مدل نیز با اعتبارسنجی متقابل اندازه‌گیری شده است.

۴. **مدل‌سازی با اشتراک دو روش (ویژگی‌های نهایی)**

- در این بخش، فقط ویژگی‌هایی که هم توسط لاسو و هم توسط RFE انتخاب شده‌اند، استفاده شده است.
- این ویژگی‌ها به‌عنوان ترکیب نهایی انتخاب ویژگی در نظر گرفته شده‌اند و مدل نهایی با استفاده از آن‌ها ساخته شده است.

خروجی:

بالاترین دقت مربوط به ویژگی‌های انتخاب‌شده توسط لاسو است (حدود ۸۰.۸ درصد). این نشان می‌دهد که لاسو توانسته ویژگی‌های مؤثرتری برای طبقه‌بندی انتخاب کند.

ویژگی‌های انتخاب‌شده توسط RFE و اشتراک بین دو روش، دقت یکسانی دارند)حدود ۷۹.۴ درصد.(

با وجود اینکه روش ترکیبی در برخی مواقع برای کاهش پیچیدگی مدل مفید است، در این مورد **عملکرد مدل نهایی به خوبی مدل بر اساس ویژگی‌های لاسو نیست.**

```python
# مدل رگرسیون لجستیک
log_reg = LogisticRegression(solver='liblinear')

# انتخاب ویژگی‌ها
X_lasso = X[lasso_selected_features]  # ویژگی‌های انتخاب شده توسط لاسو
X_rfe = X[rfe_selected_features]  # ویژگی‌های انتخاب شده توسط RFE
X_final = X[final_selected_features]  # ویژگی‌های انتخاب شده توسط هر دو روش

# تقسیم داده‌ها به داده‌های آموزشی و تست
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# آموزش مدل با ویژگی‌های انتخابی توسط لاسو
log_reg.fit(X_train, y_train)
y_pred_lasso = log_reg.predict(X_test)
accuracy_lasso = accuracy_score(y_test, y_pred_lasso)

# آموزش مدل با ویژگی‌های انتخابی توسط RFE
log_reg.fit(X_train, y_train)
y_pred_rfe = log_reg.predict(X_test)
accuracy_rfe = accuracy_score(y_test, y_pred_rfe)

# آموزش مدل با ویژگی‌های انتخابی توسط هر دو روش
log_reg.fit(X_train, y_train)
y_pred_final = log_reg.predict(X_test)
accuracy_final = accuracy_score(y_test, y_pred_final)

# محاسبه AUC و ROC برای هر مدل
fpr_lasso, tpr_lasso, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_lasso = auc(fpr_lasso, tpr_lasso)

fpr_rfe, tpr_rfe, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_rfe = auc(fpr_rfe, tpr_rfe)

fpr_final, tpr_final, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_final = auc(fpr_final, tpr_final)

# رسم نمودار ROC
plt.figure(figsize=(10, 6))
plt.plot(fpr_lasso, tpr_lasso, color='blue', lw=2, label=f'Lasso (AUC = {roc_auc_lasso:.2f})')
plt.plot(fpr_rfe, tpr_rfe, color='green', lw=2, label=f'RFE (AUC = {roc_auc_rfe:.2f})')
plt.plot(fpr_final, tpr_final, color='red', lw=2, label=f'Final (AUC = {roc_auc_final:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()

# نمایش دقت‌ها
print(f"Accuracy with Lasso selected features: {accuracy_lasso:.4f}")
print(f"Accuracy with RFE selected features: {accuracy_rfe:.4f}")
print(f"Accuracy with final selected features: {accuracy_final:.4f}")
```

رگرسیون چندکلاسه و یکی در مقابل همه

کد:

```
# تقسیم داده‌ها به مجموعه آموزش و تست
df["FamilySizeSurvivalRate"].fillna(df["FamilySizeSurvivalRate"].mode()[0], inplace=True)
y = df['FamilySizeSurvivalRate']
X = df.drop(columns=['AgeGroup','Ticket','FamilySizeSurvivalRate','AloneSurvivalRate','AgeGroupSurvivalRate',])
#print(y.isnull().sum())
#print("****")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print(y)

# 1. One-vs-Rest Logistic Regression
ovr_model = LogisticRegression(multi_class='ovr', solver='liblinear')
ovr_model.fit(X_train, y_train)
print('+++')
# پیش‌بینی با مدل One-vs-Rest
y_pred_ovr = ovr_model.predict(X_test)
accuracy_ovr = accuracy_score(y_test, y_pred_ovr)

# 2. Multinomial Logistic Regression
multi_model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
multi_model.fit(X_train, y_train)
coefficients_multinomial = multi_model.coef_
intercept_multinomial = multi_model.intercept_

print("Multinomial Logistic Regression Coefficients:")
print("Coefficients:", coefficients_multinomial)
print("Intercept:", intercept_multinomial)

# پیش‌بینی با مدل Multinomial
y_pred_multi = multi_model.predict(X_test)
accuracy_multi = accuracy_score(y_test, y_pred_multi)

# نمایش نتایج
print(f"Accuracy of One-vs-Rest Logistic Regression: {accuracy_ovr * 100:.2f}%")
print(f"Accuracy of Multinomial Logistic Regression: {accuracy_multi * 100:.2f}%")
```

توضیح:

در ابتدا، مقادیر گمشده ستون FamilySizeSurvivalRate با مقدار پرتکرار آن (مد) پر شده‌اند. سپس متغیر هدف برابر با همین ستون قرار گرفته است.

ویژگی‌های غیرضروری و دارای اطلاعات تکراری مانند گروه سنی، شماره بلیط و نرخ بقا بر اساس سایر شاخص‌ها از مجموعه داده حذف شده‌اند تا تنها ویژگی‌های مؤثر باقی بمانند.

سپس داده‌ها به دو بخش آموزش (۷۰ درصد) و آزمون (۳۰ درصد) تقسیم شده‌اند، با استفاده از مقدار تصادفی ثابت برای تکرارپذیری.

۲: مدل‌سازی با رگرسیون لجستیک One-vs-Rest

مدل اول بر اساس روش One-vs-Rest ساخته شده است. در این روش، اگر مسئله دارای چند کلاس باشد، برای هر کلاس یک مدل باینری آموزش داده می‌شود که آن کلاس را در مقابل سایر کلاس‌ها پیش‌بینی می‌کند. پس از آموزش مدل بر داده‌های آموزشی، پیش‌بینی روی داده‌های تست انجام شده و دقت مدل محاسبه شده است.

۳: مدل‌سازی با رگرسیون لجستیک Multinomial

در روش دوم، مدل رگرسیون لجستیک چندکلاسه (Multinomial) با استفاده از حل‌گر متفاوت آموزش داده شده است.
برخلاف One-vs-Rest که کلاس‌ها را به صورت جداگانه در نظر می‌گیرد، در این روش مدل به صورت هم‌زمان تمام کلاس‌ها را در نظر می‌گیرد و احتمال عضویت در هر کلاس را با توجه به سایر کلاس‌ها بهینه‌سازی می‌کند.

پس از آموزش، ضرایب و عرض از مبدأ مدل استخراج شده‌اند تا تحلیل‌پذیری مدل بهبود یابد. سپس پیش‌بینی بر روی داده‌های آزمون انجام شده و دقت مدل دوم نیز محاسبه شده است

رگرسیون دودویی:

```
y = df['FamilySizeSurvivalRate']
X = df.drop(columns=['AgeGroup','Ticket','FamilySizeSurvivalRate','AloneSurvivalRate','AgeGroupSurvivalRate',])
y = (df.FamilySizeSurvivalRate == 0).astype(int)  # تبدیل به دو کلاس (0 یا 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# مدل رگرسیون لجستیک دودویی
model_binary = LogisticRegression(solver='liblinear')

# آموزش مدل
model_binary.fit(X_train, y_train)

# محاسبه ضرایب مدل
coefficients_binary = model_binary.coef_
intercept_binary = model_binary.intercept_

print("Binary Logistic Regression Coefficients:")
print("Coefficients:", coefficients_binary)
print("Intercept:", intercept_binary)
```

```
Binary Logistic Regression Coefficients:
Coefficients: [[-1.69553912 -0.59591695 -0.74249158 -0.1688623   0.36637244  0.28233657
  -0.83476841  1.23945844 -0.13445588 -0.60511969 -0.13227368 -0.28693067
  -0.22802525 -0.24287642 -0.21876105 -0.00172197  0.15157679  0.19751014
  -0.99044343 -0.01244617  0.19751014 -0.99044343]]
Intercept: [-1.75143571]
```

خروجی:

```
coeff_df_binary
```

	Coefficient
Survived	-1.695539
Pclass	-0.595917
Age	-0.742492
SibSp	-0.168862
Parch	0.366372
Fare	0.282337
Sex_male	-0.834768
Embarked_Q	1.239458
Embarked_S	-0.134456
Cabin_B	-0.605120
Cabin_C	-0.132274
Cabin_D	-0.286931
Cabin_E	-0.228025
Cabin_F	-0.242876
Cabin_G	-0.218761
Cabin_T	-0.001722
Cabin_Unknown	0.151577
FamilySize	0.197510
IsAlone	-0.990443
Age_Fare_Ratio	-0.012446
Family_Size	0.197510
Is_Alone	-0.990443

محاسبه شانس:

```
]: # محاسبه نسبت شانس (Odds Ratio) برای هر ویژگی
odds_ratios_binary = np.exp(coefficients_binary)
# اضافه کردن نسبت شانس به DataFrame
odds_ratios_df_binary = pd.DataFrame(odds_ratios_binary.T, index=features, columns=['ODD RATio'])
odds_ratios_df_binary
```

]:

	ODD RATio
Survived	0.183500
Pclass	0.551057
Age	0.475927
SibSp	0.844625
Parch	1.442492
Fare	1.326225
Sex_male	0.433975
Embarked_Q	3.453743
Embarked_S	0.874191
Cabin_B	0.546009
Cabin_C	0.876101
Cabin_D	0.750564
Cabin_E	0.796104
Cabin_F	0.784368
Cabin_G	0.803514
Cabin_T	0.998280
Cabin_Unknown	1.163668
FamilySize	1.218365
IsAlone	0.371412
Age_Fare_Ratio	0.987631
Family_Size	1.218365
Is_Alone	0.371412

توضیح:

ویژگی	Odds Ratio	تفسیر
Survived	0.1835	مقدار مرجع، جهت مدل‌سازی بدون تفسیر مستقل
Pclass	0.5511	کلاس پایین‌تر با احتمال کمتر بقا همراه است

Age	0.4759	افزایش سن باعث کاهش احتمال بقا می‌شود
SibSp	0.8446	تعداد خواهر/برادر یا همسر همراه تأثیر کمی منفی دارد
Parch	1.4425	داشتن والدین یا فرزند همراه احتمال بقا را افزایش می‌دهد
Fare	1.3262	کرایه بالاتر با احتمال نجات بیشتر همراه است
Sex_male	0.4340	مرد بودن باعث کاهش احتمال نجات می‌شود
Embarked_Q	3.4537	سوار شدن از بندر Q احتمال نجات را به طور قابل توجهی افزایش داده است
Embarked_S	0.8742	تأثیر کم و نسبتاً منفی بر بقا
Cabin_B	0.5460	مسافران این کابین شانس کمتری برای نجات داشته‌اند
Cabin_C	0.8761	تأثیر کمی منفی و نزدیک به خنثی
Cabin_D	0.7506	کاهش متوسط در احتمال نجات
Cabin_E	0.7961	کاهش متوسط در احتمال نجات
Cabin_F	0.7844	کاهش متوسط در احتمال نجات
Cabin_G	0.8035	کاهش متوسط در احتمال نجات
Cabin_T	0.9983	تقریباً بی‌تأثیر
Cabin_Unknown	1.1637	نداشتن اطلاعات کابین احتمال نجات را کمی افزایش داده است
FamilySize	1.2184	داشتن خانواده همراه احتمال نجات را افزایش داده است
IsAlone	0.3714	سفر به‌تنهایی با کاهش احتمال بقا همراه بوده است
Age_Fare_Ratio	0.9876	تأثیر بسیار جزئی و نزدیک به صفر
Family_Size	1.2184	معادل با FamilySize مقدار تکراری)
Is_Alone	0.3714	معادل با IsAlone مقدار تکراری

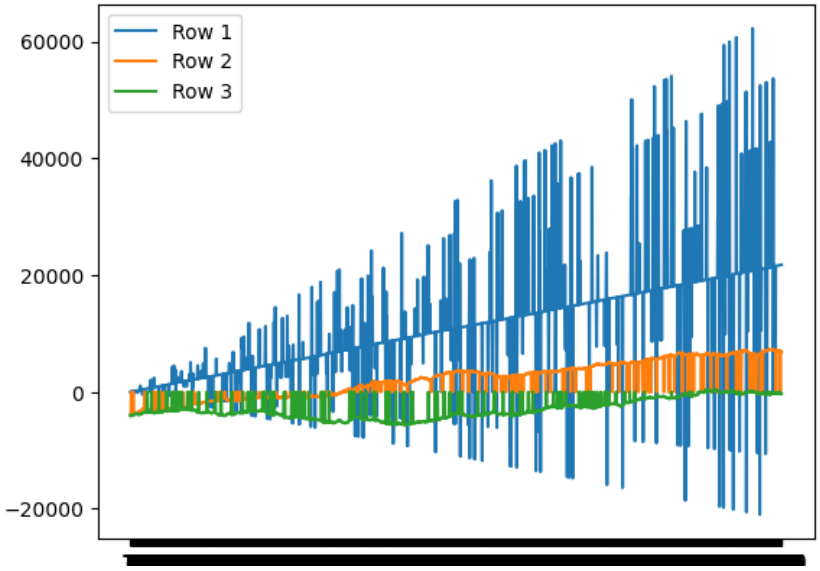
پرسش دوم دیتاست دیگر)

```
# رسم هر ردیف با رنگ‌های مختلف
for i in range(df.shape[0]):
    plt.plot(df.columns, df.iloc[i], label=f'Row {i+1}')

# برچسب زدن به محورها و اضافه کردن شرح

plt.legend()

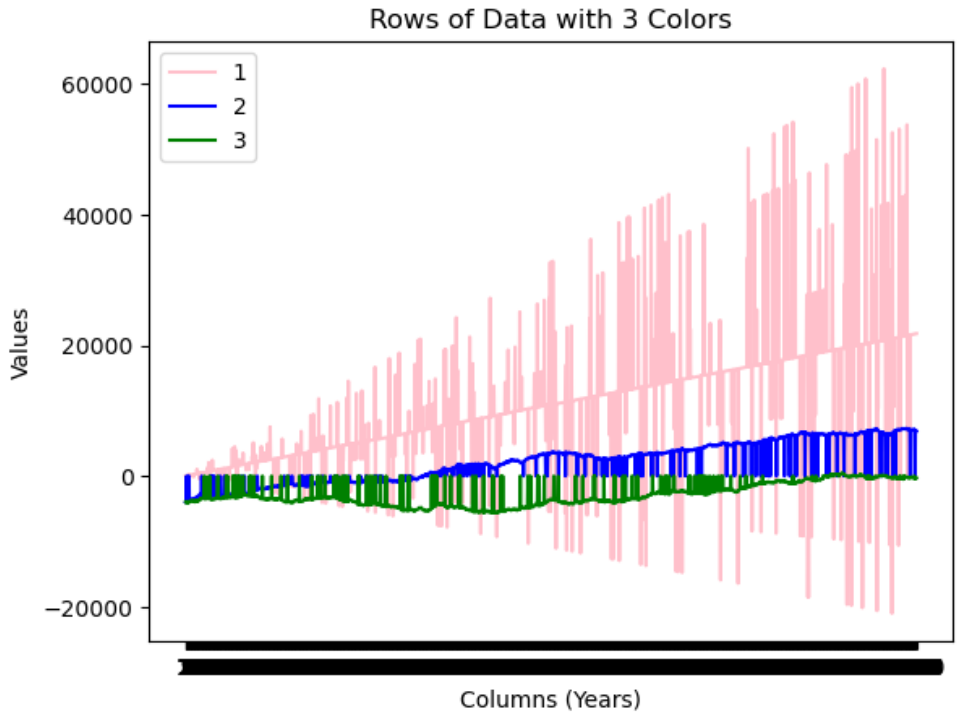
# نمایش نمودار
plt.show()
```



دیف اول (به رنگ آبی) دارای بیشترین نوسانات و دامنه‌ی تغییرات بسیار گسترده‌ای در طول بازه‌ی زمانی مورد بررسی است، به‌گونه‌ای که مقادیر آن به طور قابل توجهی افزایش یافته و واریانس بالایی از خود نشان می‌دهد. این رفتار می‌تواند ناشی از تأثیرگذاری یک عامل افزایشی قوی یا وجود نویز شدید در داده‌ها باشد .ردیف دوم (به رنگ نارنجی) نسبت به ردیف اول دارای تغییرات بسیار ملایم‌تری است و با وجود برخی نوسانات، روند کلی آن افزایشی بوده و از ثبات نسبی برخوردار است. این امر می‌تواند حاکی از کنترل بهتر متغیرهای مؤثر بر این ردیف باشد .در نهایت، ردیف سوم (به رنگ سبز) تقریباً در محور افقی نوسان دارد و تغییرات آن محدود و نسبتاً پایدار است. داده‌های این ردیف از روند خاصی پیروی نمی‌کنند و نوسانات آن بیشتر حول مقدار صفر متمرکز شده است.

```
colors=['pink','blue','green']
for i ,row in df.iterrows():
  plt.plot(row.index,row.values, label=f'{i+1}',color=colors[i])

plt.xlabel('Columns (Years)')
plt.ylabel('Values')
plt.title('Rows of Data with 3 Colors')
plt.legend()
plt.show()
```



تحلیل:

مشاهدات از نمودار:

۱. **ردیف اول (رنگ صورتی):**

- مقادیر بسیار بزرگ دارد (حتی تا حدود ۶۰،۰۰۰).
- روند صعودی آشکار در محور y دارد.
- نوسانات زیادی نیز دارد (دامنه بالا).

۲. **ردیف دوم (رنگ آبی):**

- مقدارهای پایین‌تر نسبت به ردیف اول.
- نوسان دارد اما بسیار ملایم‌تر از ردیف اول است.

- یک روند افزایشی ملایم در سال‌ها مشاهده می‌شود.

۳. **ردیف سوم (رنگ سبز):**

- مقدارها در بازه‌ای نزدیک به صفر قرار دارند.
- نوسانات نسبتاً کم و کم‌دامنه دارد.
- به نظر می‌رسد روند خاصی (صعودی یا نزولی) در آن وجود ندارد.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
# آماده‌سازی داده‌ها  برای رگرسیون (اضافه کردن ستون 1 برای با یاس)
X_cleaned_no_outliers = np.c_[np.ones(len(row_1_no_outliers)), row_1_no_outliers]
X_cleaned_with_imputation = np.c_[np.ones(len(row_1_with_imputation)), row_1_with_imputation]

# در محاسبات NaN مقیاس‌بندی داده‌ها  برای جلوگیری از
#scaler = StandardScaler()

# مقیاس‌بندی داده‌های پاکسازی‌شده (حذف داده‌های پرت و جایگزینی با میانگین همسایه‌ها)
X_scaled_no_outliers = scaler.fit_transform(X_cleaned_no_outliers[:, 1].reshape(-1, 1))  # مقیاس‌بندی ویژگی
X_scaled_with_imputation = scaler.fit_transform(X_cleaned_with_imputation[:, 1].reshape(-1, 1))  # مقیاس‌بندی ویژگی

# اضافه کردن ستون 1 برای با یاس
X_scaled_no_outliers = np.c_[np.ones(X_scaled_no_outliers.shape[0]), X_scaled_no_outliers]
X_scaled_with_imputation = np.c_[np.ones(X_scaled_with_imputation.shape[0]), X_scaled_with_imputation]

# مدل رگرسیون خطی از صفر (با  استفاده از معادله عادی)
def linear_regression_model(X, y):
    # حل معادله عادی برای به دست آوردن theta
    theta = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
    return theta

# آماده‌سازی مقادیر هدف برای رگرسیون
y_cleaned_no_outliers = row_1_no_outliers.values
y_cleaned_with_imputation = row_1_with_imputation.values

# مدل‌ها  با  استفاده  از معادله عادی
theta_no_outliers = linear_regression_model(X_scaled_no_outliers, y_cleaned_no_outliers)
theta_imputation = linear_regression_model(X_scaled_with_imputation, y_cleaned_with_imputation)

# چاپ پارامترهای مدل‌ها
print("Model parameters for cleaned data (No Outliers) (Normal Equation):")
print("Intercept (B0):", theta_no_outliers[0])
print("Slope (B1):", theta_no_outliers[1])

print("\nModel parameters for cleaned data (Imputation) (Normal Equation):")
print("Intercept (B0):", theta_imputation[0])
print("Slope (B1):", theta_imputation[1])

# رسم نتایج
plt.figure(figsize=(12, 6))

# رسم خطوط رگرسیون برای هر مدل
plt.subplot(1, 2, 1)
plt.scatter(X_scaled_no_outliers[:, 1], y_cleaned_no_outliers, color='blue', label='Data (No Outliers)')
plt.plot(X_scaled_no_outliers[:, 1], X_scaled_no_outliers.dot(theta_no_outliers), color='red', label='Regression Line (No Outliers)')
plt.scatter(X_scaled_with_imputation[:, 1], y_cleaned_with_imputation, color='purple', label='Data (Imputation)')
plt.plot(X_scaled_with_imputation[:, 1], X_scaled_with_imputation.dot(theta_imputation), color='orange', label='Regression Line (Imputation)')
plt.title('Linear Regression Lines (Normal Equation)')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

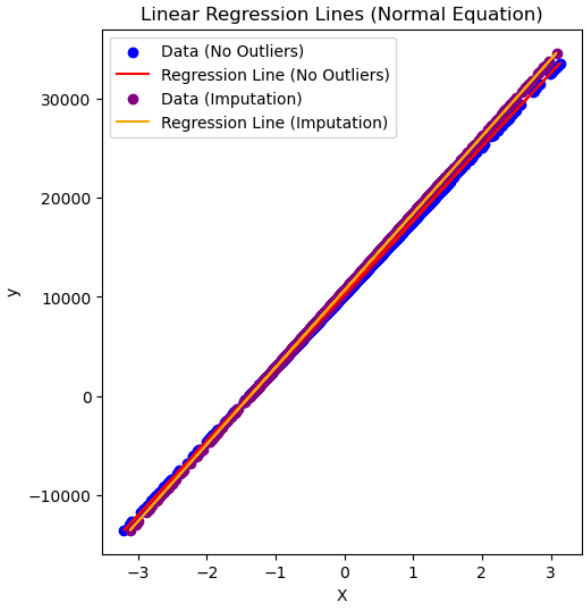
# رسم همگرایی هزینه  برای هر مدل
plt.subplot(1, 2, 2)
plt.plot(range(1000), cost_history_mse_no_outliers, label='Cost (No Outliers)')
plt.plot(range(1000), cost_history_mse_imputation, label='Cost (Imputation)')
plt.title('Cost Convergence')
plt.xlabel('Iterations')
plt.ylabel('Cost')
plt.legend()

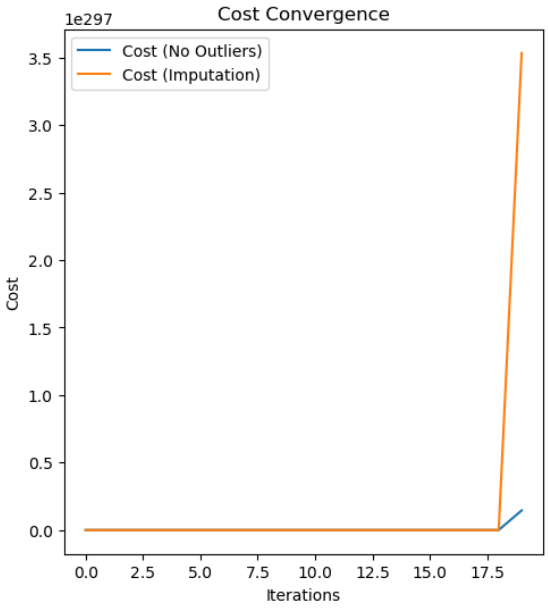
plt.show()
```

خروجی:

```
Model parameters for cleaned data (No Outliers) (Normal Equation):
Intercept (B0): 10275.64418604651
Slope (B1): 7433.43795399564

Model parameters for cleaned data (Imputation) (Normal Equation):
Intercept (B0): [10654.93583333]
Slope (B1): [7768.1481792]
```





تحلیل کد:

مقایسه ضرایب:

- **بدون داده پرت(No Outliers)**
  - Intercept (B0): حدود ۱۰۲۷۵
  - Slope (B1): حدود ۷۴۳۳
- **با داده تکمیل‌شده(Imputation)**
  - Intercept (B0): حدود ۱۰۶۵۴
  - Slope (B1): حدود ۷۷۶۸

تفاوت بین خطوط ناچیز است، که نشان می‌دهد هر دو مدل توانسته‌اند به خوبی به داده‌ها برازش پیدا کنند. هرچند مدل حاصل از داده‌های تکمیل‌شده دارای شیب کمی بالاتر است.

تابع هزینه:

تفسیر:

- مدل **No Outliers** به‌درستی همگرا شده و تابع هزینه‌اش معقول است.
- مدل **Imputation** دچار ناپایداری عددی شده است (احتمالاً به دلیل داده‌های نامتوازن یا پر شدن نادرست مقادیر گمشده)، و به همین دلیل مقدار تابع هزینه به طور انفجاری افزایش یافته است.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
row_1_no_outliers = row_1[~outliers_iqr]

# آماده‌سازی داده‌ها  برای رگرسیون (اضافه کردن ستون 1 برای بایاس)
X_original = np.c_[np.ones(len(row_1)), row_1.values]  # داده‌های اصلی (بدون پاکسازی)
X_cleaned_no_outliers = np.c_[np.ones(len(row_1_no_outliers)), row_1_no_outliers.values]  # داده‌های پاکسازی‌شده (حذف داده‌های پرت)

y_original = row_1.values  # برچسب‌های داده‌های اصلی
y_cleaned_no_outliers = row_1_no_outliers.values  # برچسب‌های داده‌های پاکسازی‌شده (حذف داده‌های پرت)


# آموزش مدل‌ها  با  داده‌های اصلی
model_original = LinearRegression()
model_original.fit(X_original[:, 1].reshape(-1, 1), y_original)

# آموزش مدل‌ها  با  داده‌های پاکسازی‌شده (حذف داده‌های پرت)
model_cleaned_no_outliers = LinearRegression()
model_cleaned_no_outliers.fit(X_cleaned_no_outliers[:, 1].reshape(-1, 1), y_cleaned_no_outliers)

# چاپ پارامترهای مدل‌ها
print("Model parameters for original data (LinearRegression):")
print("Intercept (B0):", model_original.intercept_)
print("Slope (B1):", model_original.coef_)

print("\nModel parameters for cleaned data (No Outliers) (LinearRegression):")
print("Intercept (B0):", model_cleaned_no_outliers.intercept_)
print("Slope (B1):", model_cleaned_no_outliers.coef_)

# رسم مدل‌ها
plt.figure(figsize=(12, 6))

# رسم داده‌های اصلی و مدل‌های رگرسیون
plt.subplot(1, 2, 1)
plt.scatter(X_original[:, 1], y_original, color='blue', label='Original Data')
plt.plot(X_original[:, 1], model_original.predict(X_original[:, 1].reshape(-1, 1)), color='red', label='Regression Line (Original)')

plt.scatter(X_cleaned_no_outliers[:, 1], y_cleaned_no_outliers, color='green', label='Cleaned Data (No Outliers)')
plt.plot(X_cleaned_no_outliers[:, 1], model_cleaned_no_outliers.predict(X_cleaned_no_outliers[:, 1].reshape(-1, 1)), color='orange', label='Regression

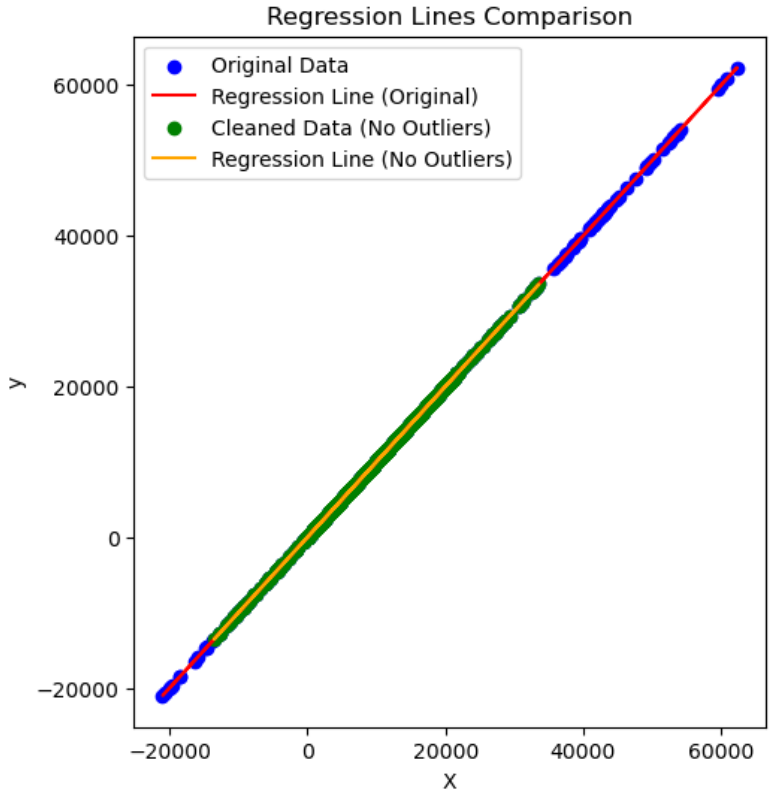
plt.title('Regression Lines Comparison')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

plt.show()
```

خروجی:

```
Model parameters for original data (LinearRegression):
Intercept (B0): 1.8189894035458565e-12
Slope (B1): [1.]

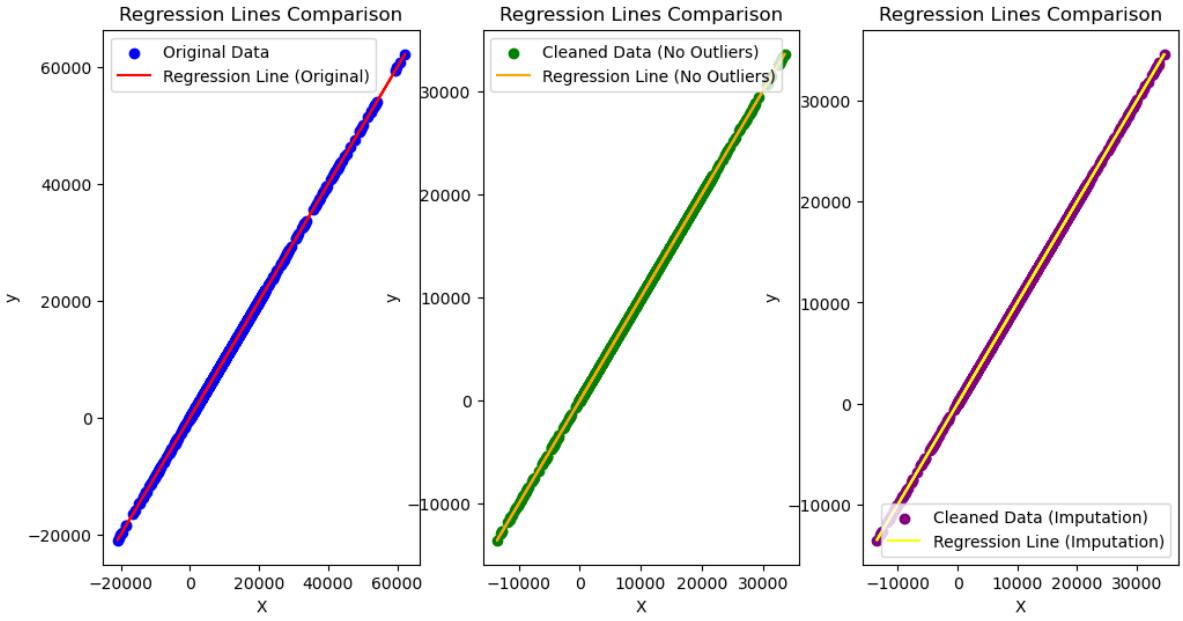
Model parameters for cleaned data (No Outliers) (LinearRegression):
Intercept (B0): 7.275957614183426e-12
Slope (B1): [1.]
```



```
Model parameters for original data (LinearRegression):
Intercept (B0): 1.8189894035458565e-12
Slope (B1): [1.]

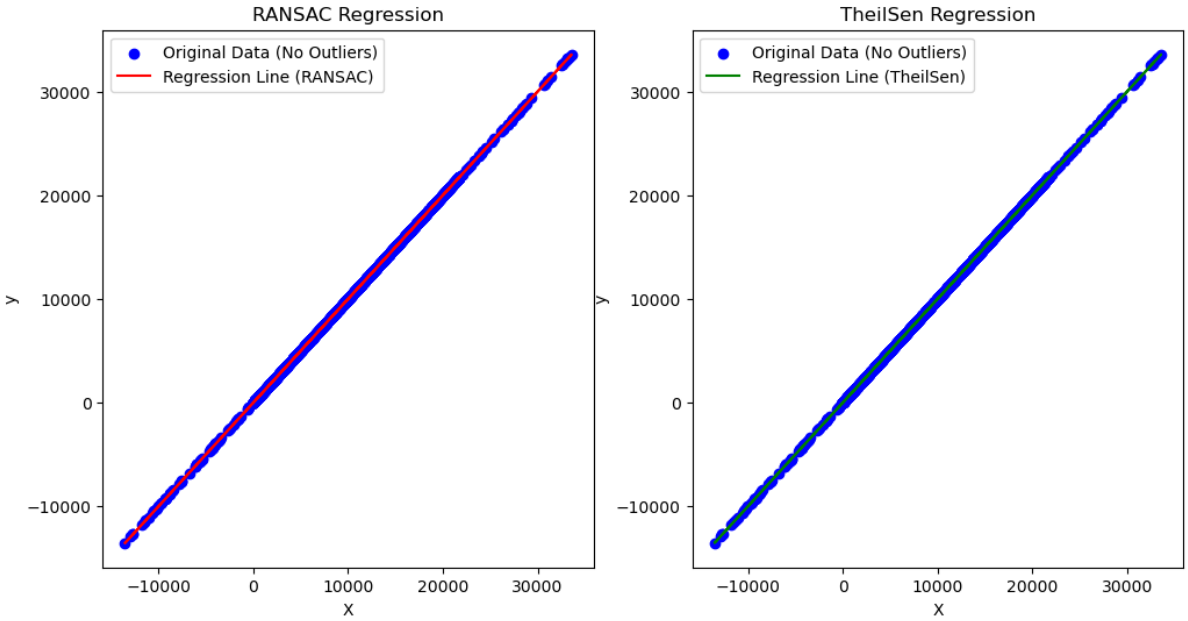
Model parameters for cleaned data (No Outliers) (LinearRegression):
Intercept (B0): 10275.644186046511
Slope (B1): [7433.437954]

Model parameters for cleaned data (Imputation) (LinearRegression):
Intercept (B0): [-5.45696821e-12]
Slope (B1): [[1.]]
```

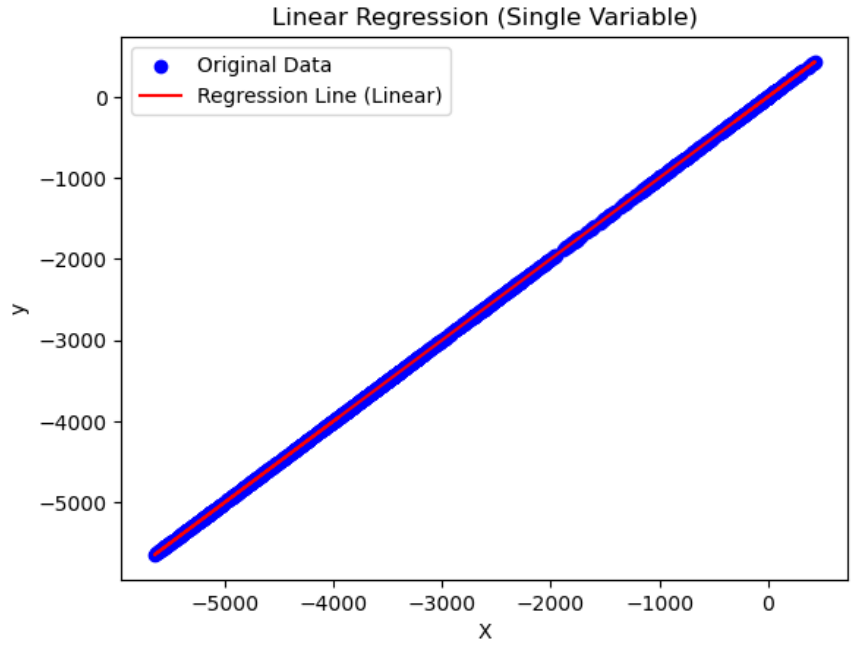


```
Model parameters for RANSAC (Robust Regression):
Intercept (B0): 7.275957614183426e-12
Slope (B1): [1.]

Model parameters for TheilSen (Robust Regression):
Intercept (B0): -5.29687158144092e-15
Slope (B1): [0.99999996]
```

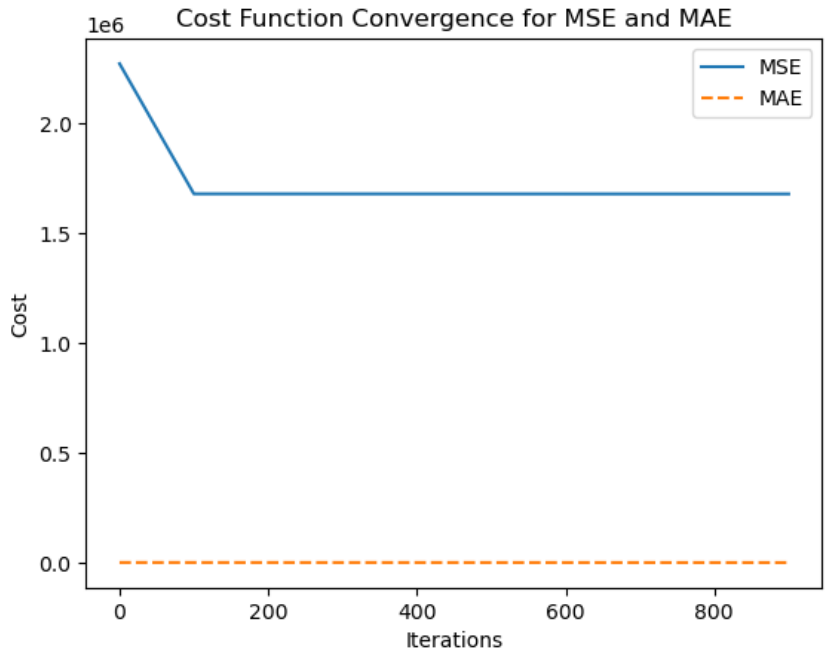


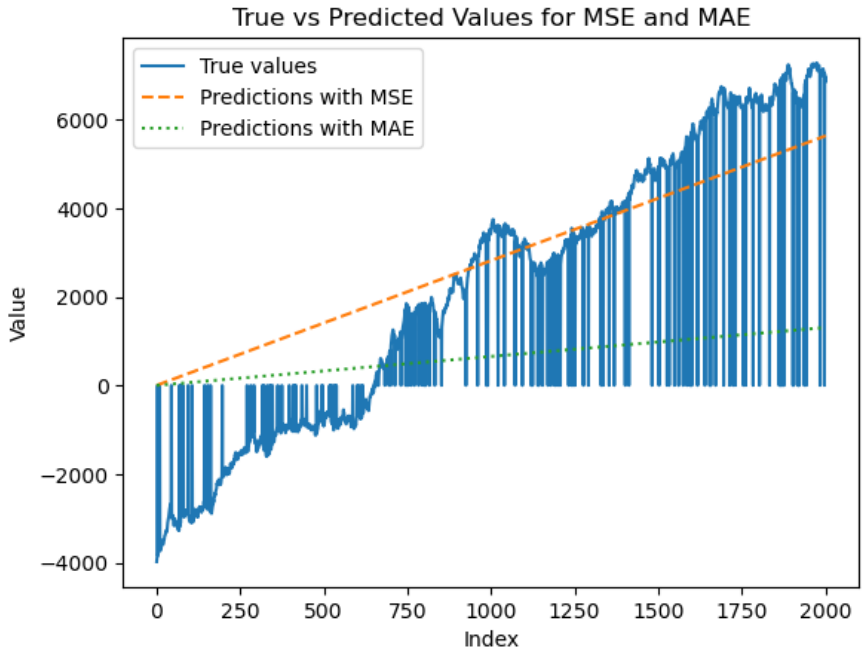

```
Model Comparison (MSE and MAE):
Linear Model MSE: 1.1009095998130243e-25, MAE: 2.5339597087281617e-13
```



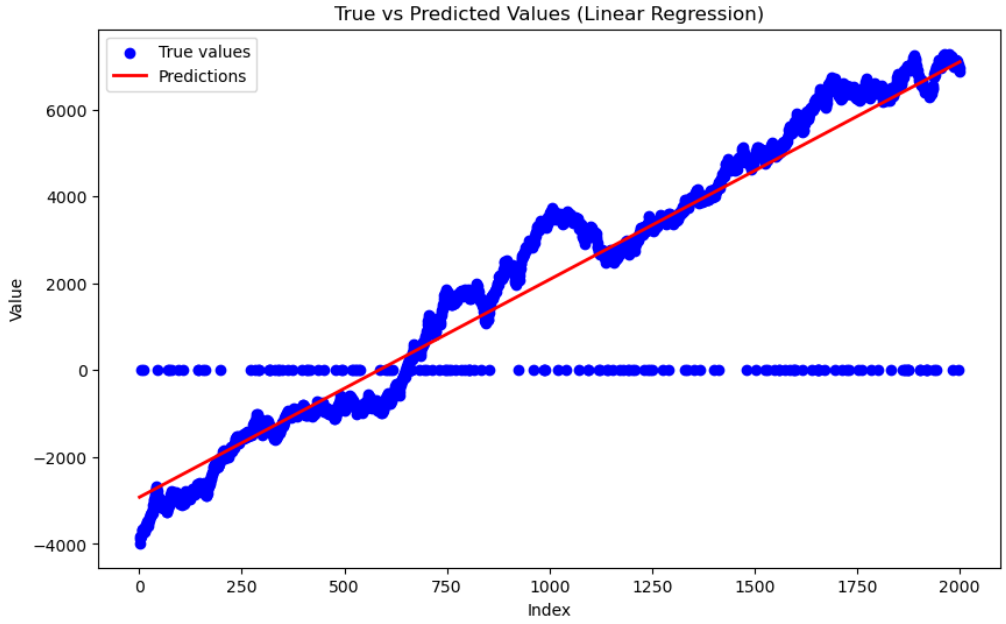
سطر دوم:

```
Final coefficients with MSE: [-0.72876692  2.81854351]
Final coefficients with MAE: [2.43580500e-04 6.53603745e-01]
```





```
]: # رسم نتایج در فضای دو بعدی
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='True values')
plt.plot(X, y_pred, color='red', label='Predictions', linewidth=2)
plt.xlabel('Index')
plt.ylabel('Value')
plt.title('True vs Predicted Values (Linear Regression)')
plt.legend()
plt.show()
```



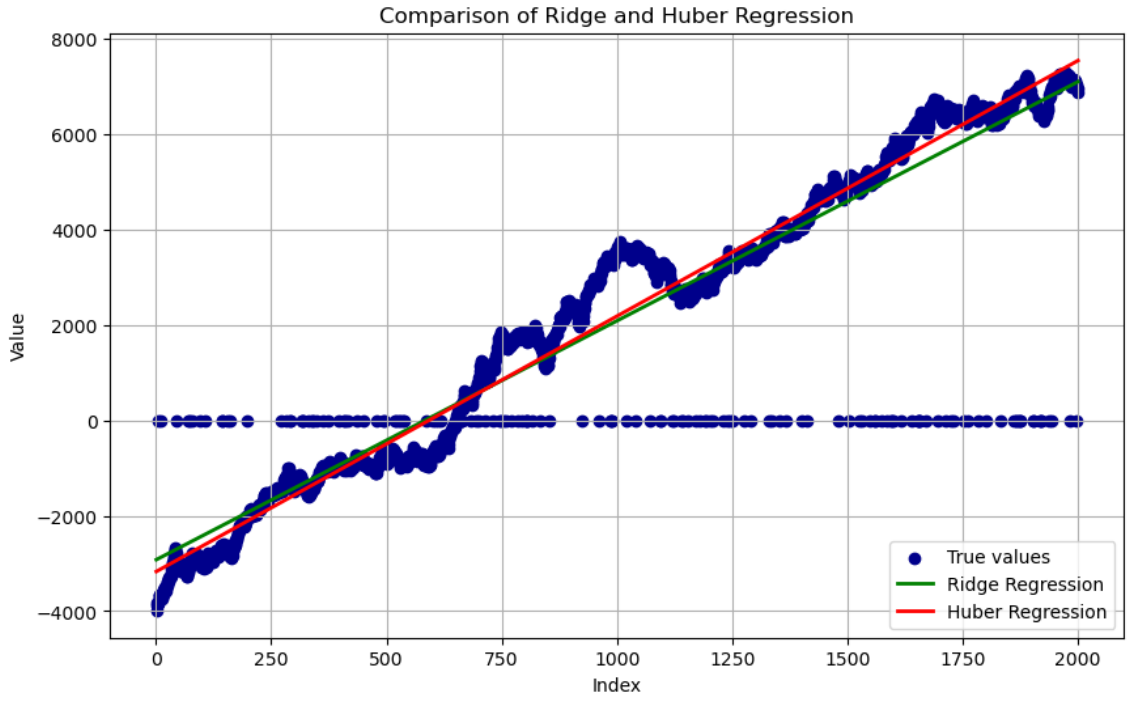
رگرسیون مقاوم:

```
# رسم نتایج در فضای دو بعدی
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='DarkBlue', label='True values')

# رسم پیش‌بینی‌ها با مدل Ridge
plt.plot(X, y_pred_ridge, color='green', label='Ridge Regression', linewidth=2)

# رسم پیش‌بینی‌ها با مدل Huber
plt.plot(X, y_pred_huber, color='red', label='Huber Regression', linewidth=2)


plt.xlabel('Index')
plt.ylabel('Value')
plt.title('Comparison of Ridge and Huber Regression')
plt.legend(loc='lower right')
plt.grid()
plt.show()
```



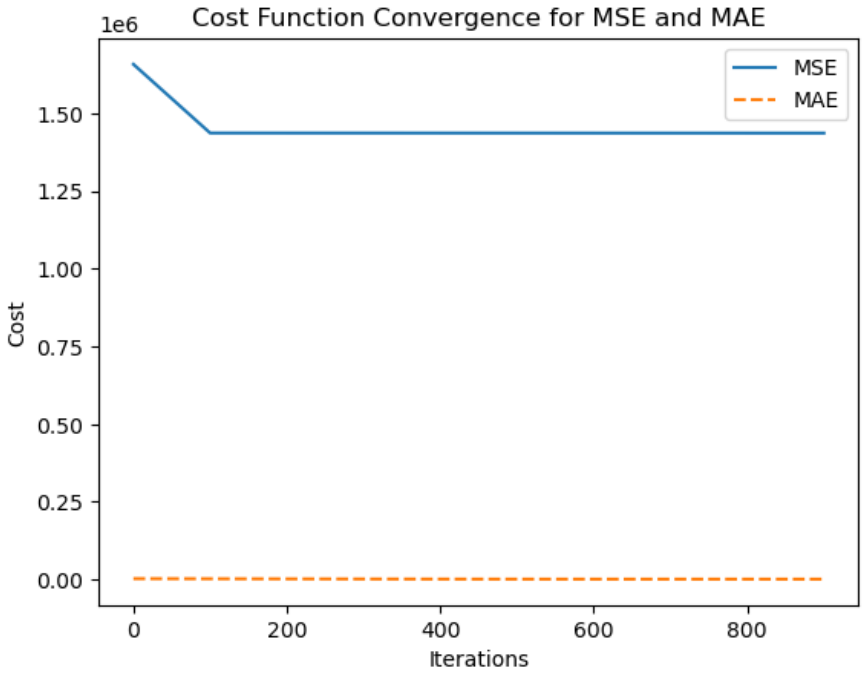
داده های پرت برای سطر دوم:

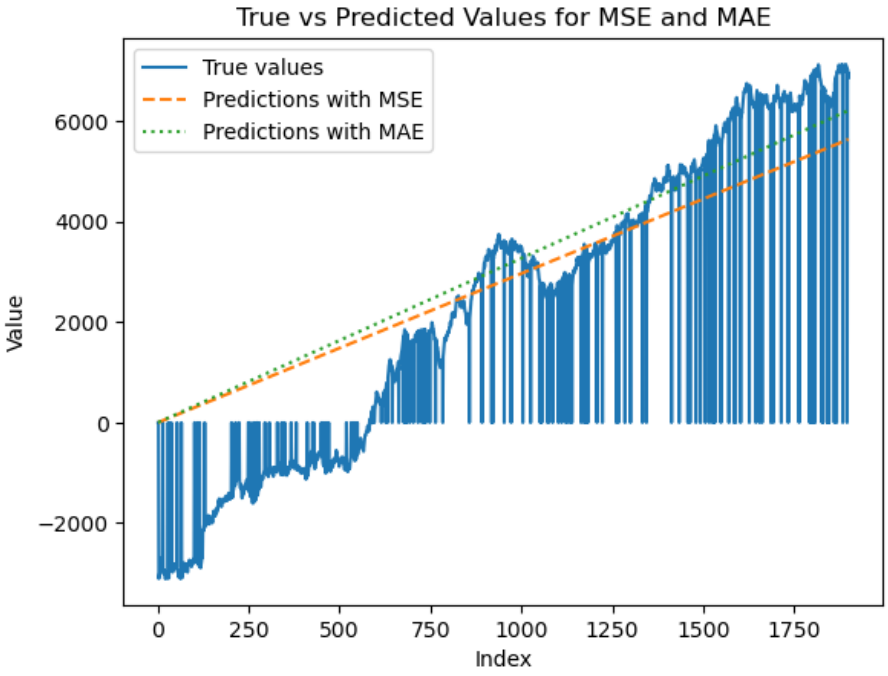
```
# استفاده از الگوریتم Isolation Forest
row2_reshaped = row2.reshape(-1, 1)
iso_forest = IsolationForest(contamination=0.05)
outliers_iforest = iso_forest.fit_predict(row2_reshaped)
```

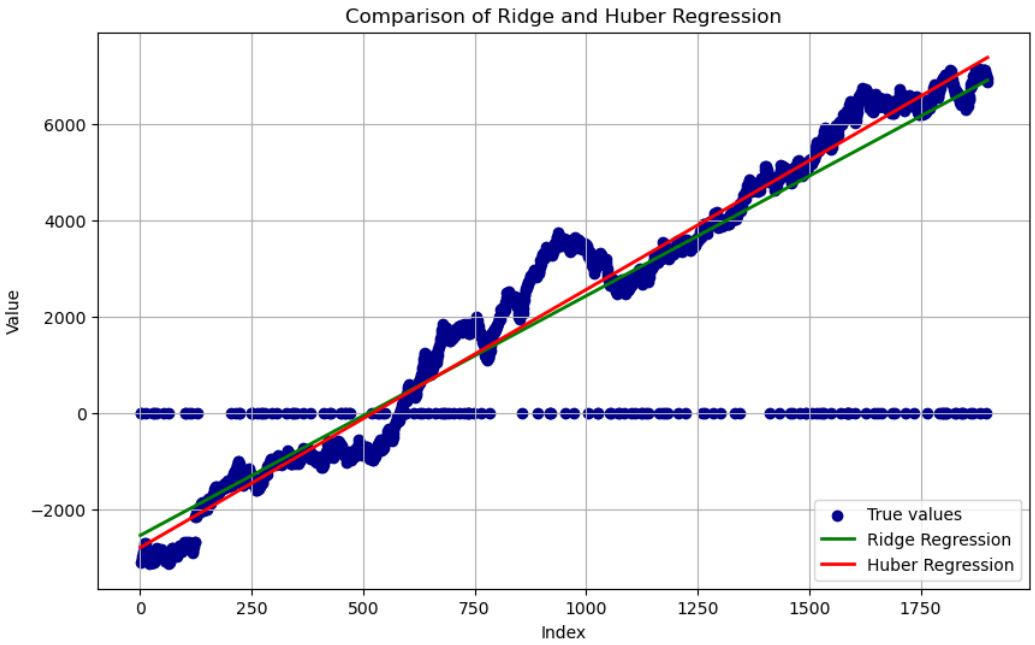
```
# شناسایی داده‌های پرت (برچسب -1 نشان‌دهنده داده‌های پرت است)
outliers = np.where(outliers_iforest == -1)[0]
outliers
```

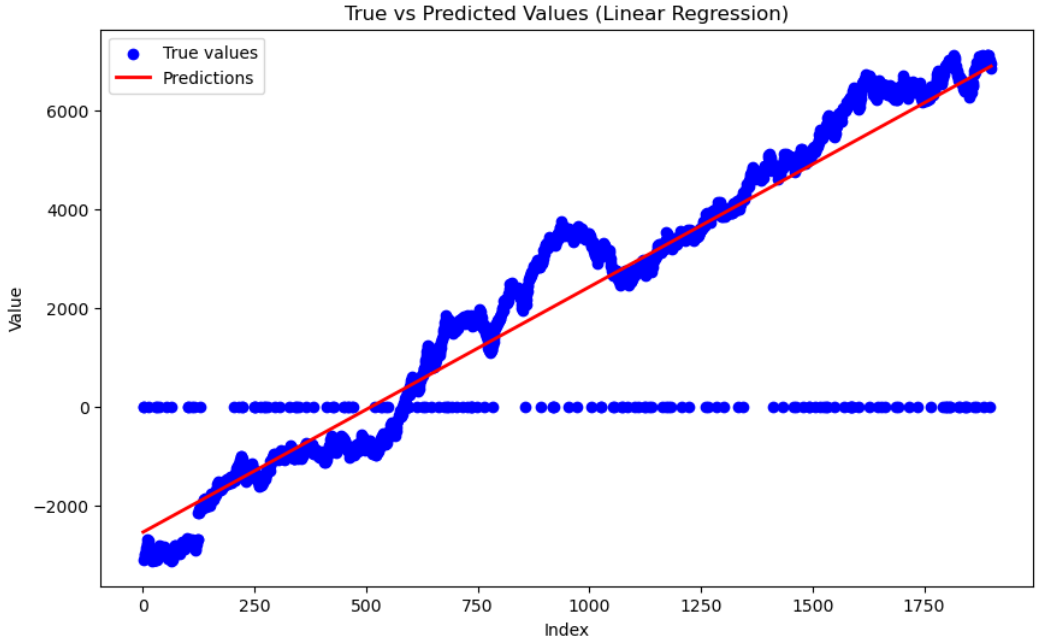
```
array([   0,    1,    2,    4,    5,    6,    7,    8,   10,   11,   12,
         13,   14,   15,   16,   17,   18,   19,   20,   21,   22,   23,
         24,   25,   26,   27,   28,   29,   30,   31,   32,   52,   59,
         61,   62,   63,   64,   65,   66,   67,   69,   70,  141,  145,
        155,  156,  157,  170,  171,  172,  173,  174,  175,  176,  177,
        178,  179,  180,  181,  182,  183,  184,  186,  188,  189,  190,
        191,  192, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1948, 1955,
       1957, 1958, 1959, 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967,
       1968, 1969, 1970, 1971, 1972, 1973, 1974, 1977, 1978, 1979, 1980,
       1991], dtype=int64)
```

```
Final coefficients with MAE: [-5.70721053e-04  3.27053457e+00]
```

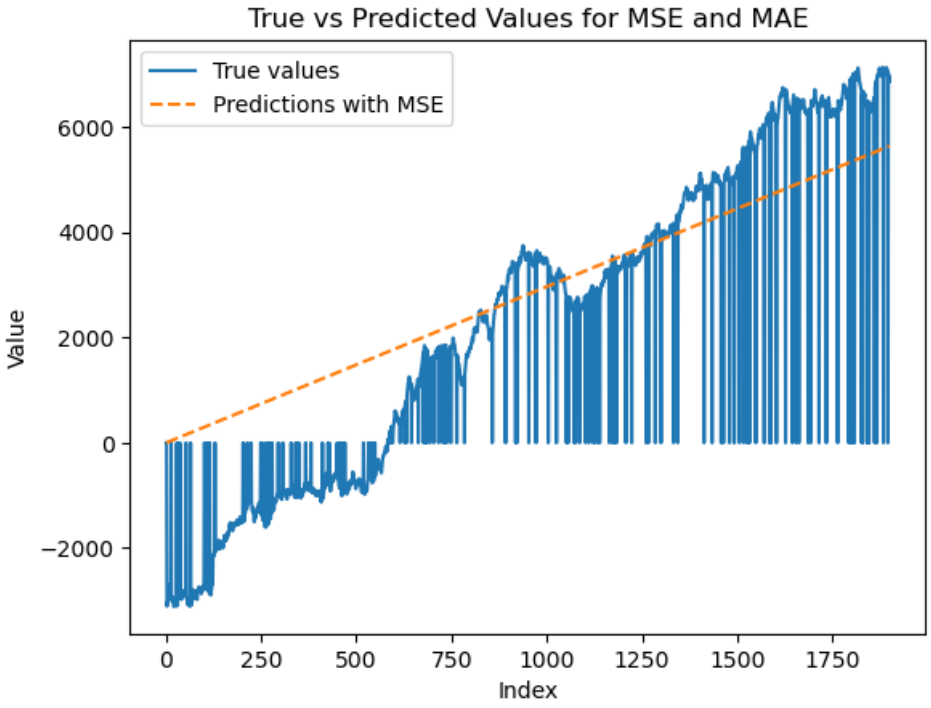


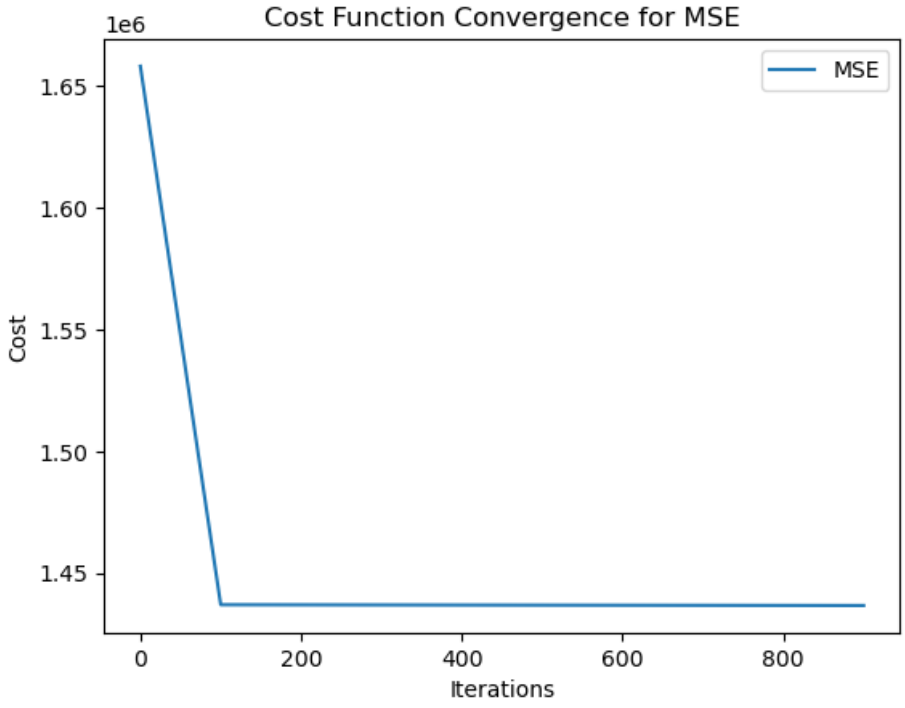



Comparison of Ridge and Huber Regression
Value
Index
True values
Ridge Regression
Huber Regression
6000
4000
2000
0
−2000
0
250
500
750
1000
1250
1500
1750


True vs Predicted Values (Linear Regression)
Value
Index
True values
Predictions
6000
4000
2000
0
−2000
0
250
500
750
1000
1250
1500
1750

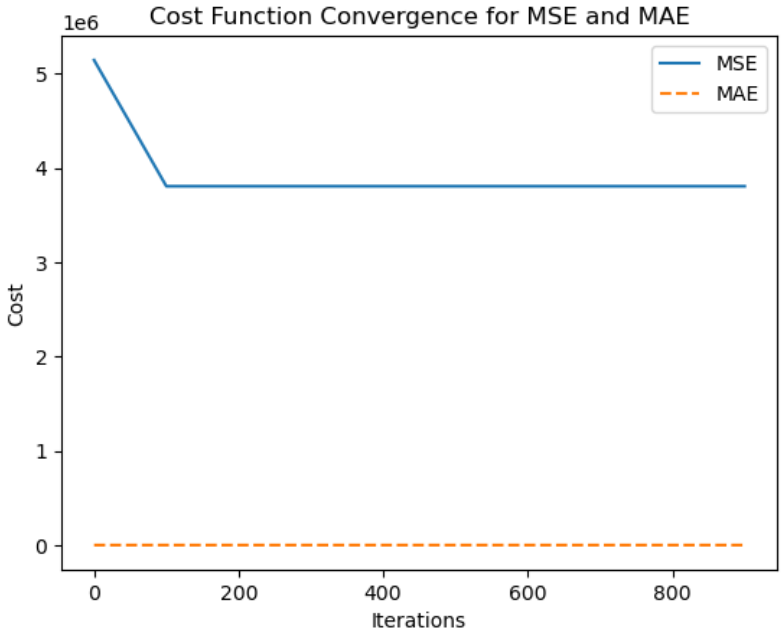
```
Final coefficients with MSE: [-0.6309616   2.96702293]
```

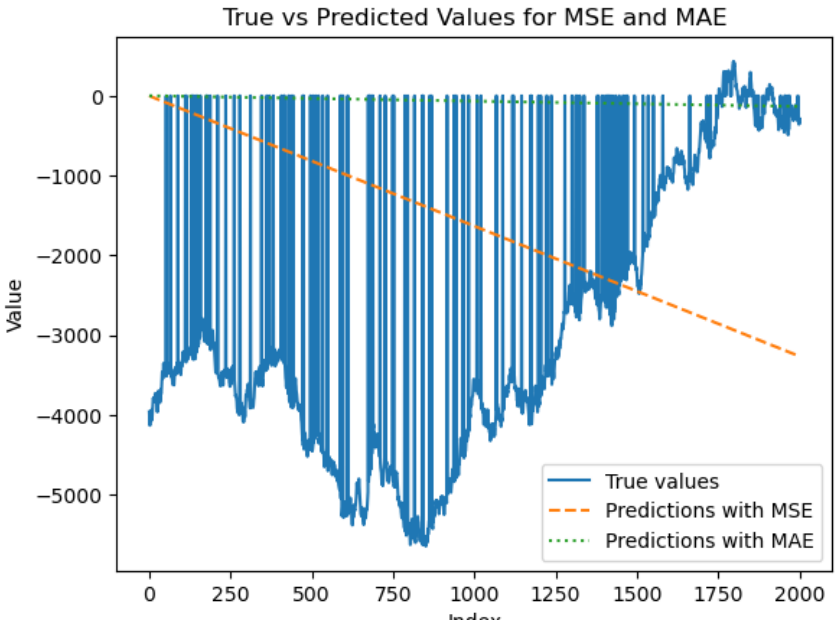




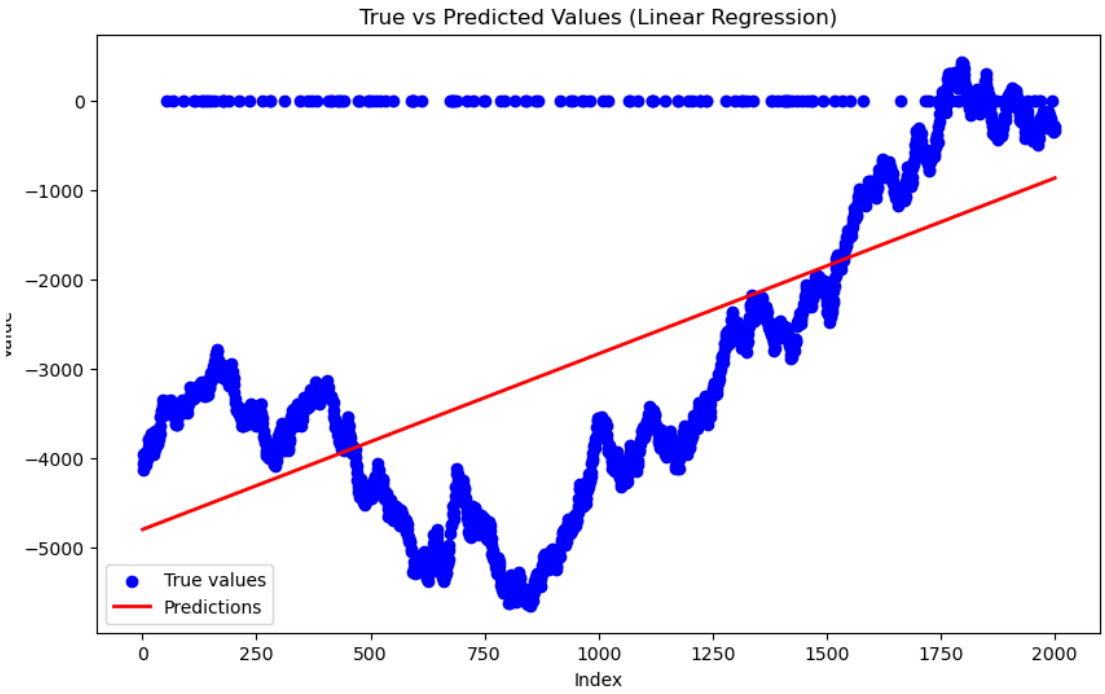
سطر سوم نیز به همین منوال با استفاده از روش ایزو فارست داده های پرتش شناسایی شده و مراحل قبل روی ان تکرار شده است اما با دقت بسیار پایین نتیجه داده است.

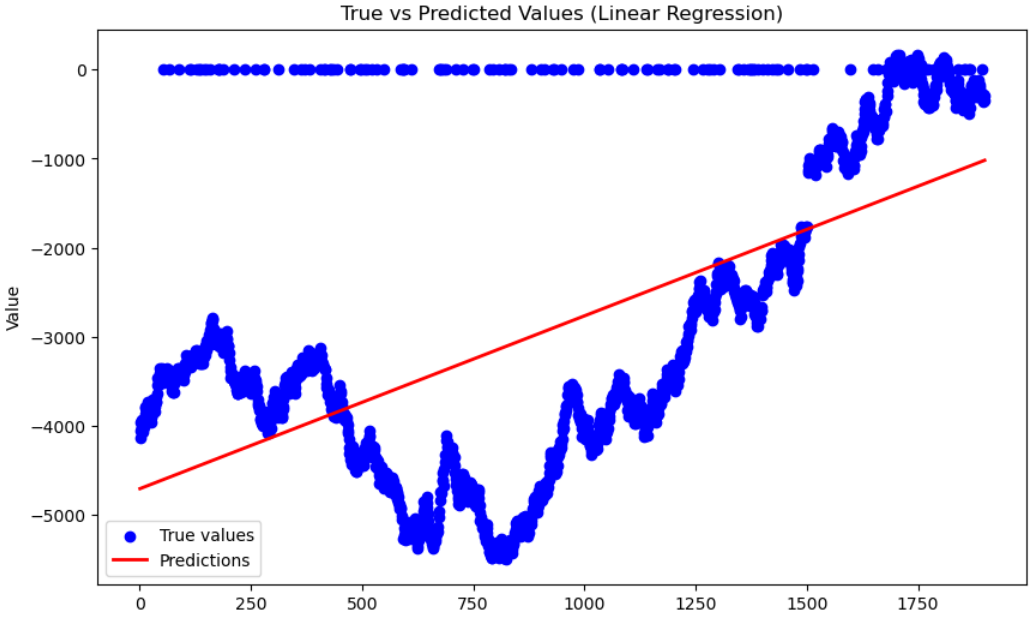
```
Final coefficients with MSE: [-0.12106707 -1.63294943]
Final coefficients with MAE: [-7.57316000e-05 -6.69569725e-02]
```

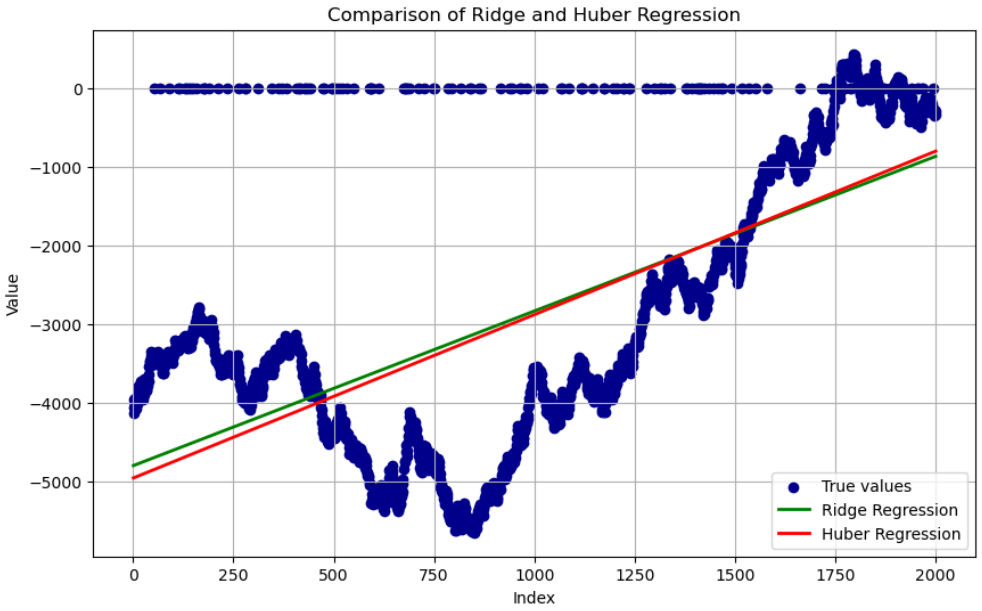




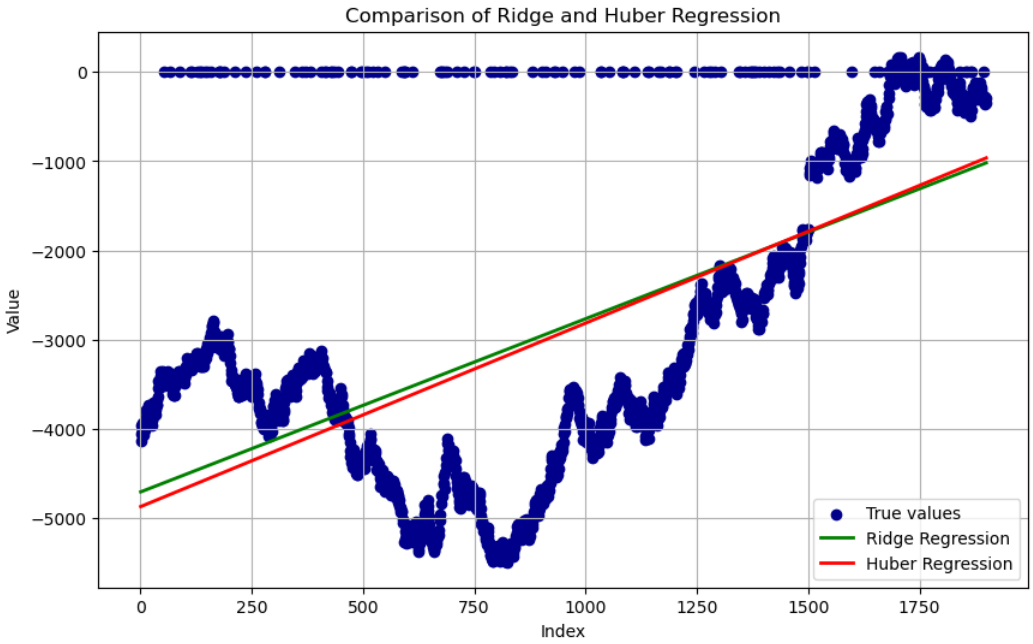
اختلاف پیشبینی و واقعیت بسیار زیاد است


True vs Predicted Values (Linear Regression)
0
−1000
−2000
−3000
−4000
−5000
True values
Predictions
0
250
500
750
1000
1250
1500
1750
2000
Index


True vs Predicted Values (Linear Regression)
0
−1000
−2000
−3000
−4000
−5000
Value
True values
Predictions
0
250
500
750
1000
1250
1500
1750




```
X2 = np.array([np.arange(1, len(row3_cleaned) + 1)]).T
y2 = row3_cleaned
plt.show()
```



سوال امتیازی:

تفاوت تابع اتلاف و تابع هزینه را بیان کنید.

در حوزه یادگیری ماشین، دو مفهوم «تابع خطا (Loss Function)» و «تابع هزینه (Cost Function)» «کاربرد فراوانی دارند و در برخی منابع ممکن است به صورت مترادف به کار روند؛ با این حال، میان آن‌ها تفاوت مفهومی مشخصی وجود دارد.

تابع اتلاف: تابع اتلاف، خطای مدل را برای یک نمونه خاص از داده اندازه‌گیری می‌کند. این تابع نشان می‌دهد که پیش‌بینی مدل برای یک داده‌ی ورودی چقدر با مقدار واقعی تفاوت دارد.

$$Loss = \mathcal{L}(\hat{y}, y)$$

$\hat{y}$: پیش بینی مدل , y: مقدار واقعی

تابع هزینه: تابع هزینه معمولاً به میانگین تابع اتلاف روی کل مجموعه داده گفته می‌شود. یعنی ارزیابی عملکرد کلی مدل بر اساس کل داده‌های موجود، که فرمول کلی آن به صورت زیر است:

$$cost = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

	تابع اتلاف $(Loss)$	تابع هزینه $(Cost)$
دامنه کاربرد	فقط روی یک نمونه	روی کل مجموعه داده
استفاده در آموزش	برای محاسبه خطای لحظه‌ای	برای بهینه سازی کلی مدل
محاسبه	مقدار عددی برای هر داده	میانگین یا مجموع اتلاف‌ها

مدل‌های شما چگونه اختلاف بین پیشبینی و مقدار واقعی را کمینه می‌کنند؟

رگرسیون لجستیک دوتایی[1]

در این مدل، هدف پیش‌بینی یک متغیر وابسته دوتایی است. برای این منظور، از تابع لاجیت (سیگموید) جهت تبدیل خروجی خطی به احتمال استفاده می‌شود. مدل با کمینه‌سازی تابع زیان آنتروپی متقاطع[2] آموزش می‌بیند.

رگرسیون لجستیک چندکلاسه[3]

این مدل برای طبقه‌بندی بین چند دسته به‌کار می‌رود. از تابع سافت‌مکس[4] برای تولید توزیع احتمال بین کلاس‌ها استفاده می‌شود و هدف، کمینه‌سازی آنتروپی متقاطع چندکلاسه است که اختلاف بین برچسب‌های واقعی و پیش‌بینی‌شده را کاهش می‌دهد.

رگرسیون مقاوم[5]

[1] Binary Logistic Regression
[2] Cross-Entropy
[3] Multiclass Logistic Regression
[4] Softmax
[5] Robust Regression

در رگرسیون مقاوم، هدف کاهش تأثیر داده‌های پرت[6] بر مدل است. برخلاف رگرسیون معمولی که از میانگین مربعات خطا استفاده می‌کند، در این مدل از توابع زیان مقاوم مانند تابع هابر[7] استفاده می‌شود تا اثر مقادیر پرت کاهش یابد و مدل پایدارتری حاصل شود.

بررسی کنید برای این داده‌ها کدام توابع هزینه مناسب‌تر هستند.

پرسش اول:

برای این مجموعه داده که هدف پیش‌بینی وضعیت زنده ماندن افراد است، از رگرسیون لجستیک دوتایی استفاده می‌شود. از آنجا که مسئله از نوع طبقه‌بندی دوتایی است، تابع هزینه مناسب برای آن، آنتروپی متقاطع دوتایی[8] می‌باشد. این تابع، اختلاف بین برچسب واقعی و احتمال پیش‌بینی‌شده را اندازه‌گیری می‌کند و برای مسائل طبقه‌بندی بسیار رایج و مناسب است.

تعریف این تابع به صورت زیر می‌باشد:

$$\mathcal{L}(\hat{y}, y) = -[y.log(\hat{y}) + (1-y).log(1-\hat{y})]$$

پرسش دوم:

با توجه به ساختار عددی و پیوسته داده‌ها در این مجموعه، نوع مسئله رگرسیون عددی تشخیص داده شد. در حالت کلی، برای مسائل رگرسیون از میانگین مربعات خطا[9] استفاده می‌شود، زیرا این تابع نسبت به اختلاف بین مقادیر واقعی و پیش‌بینی‌شده حساس است و برای داده‌های بدون نویز زیاد مناسب است. با این حال، در صورت وجود داده‌های پرت، می‌توان از تابع هابر[10] که نسبت به مقادیر پرت مقاوم‌تر است، استفاده کرد.

تعریف کلی تابع میانگین مربعات خطا:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

تعریف کلی تابع هابر:

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & if\ |a| \le \delta \\ \delta(|a| - \frac{1}{2}\delta) & ow \end{cases}$$

[6] outliers
[7] Huber Loss
[8] Binary Cross-Entropy
[9] Mean Squared Error - MSE
[10] Huber Loss